
openPangu-2.0: Towards Reliable and Efficient Agentic Reasoning

openPangu Team

Abstract

We introduce openPangu-2.0, comprising two primary architectural variants: **openPangu-2.0-Pro** with 505B parameters (18B activated) and **openPangu-2.0-Flash** with 92B parameters (6B activated). Additionally, we present an optimized 30B-parameter on-device model (2B activated) tailored for resource-constrained execution. To maximize agentic performance and system-level efficiency, we establish a rigorous hardware-software co-design methodology that tightly couples algorithmic innovation with Ascend-native infrastructure. At the architectural level, we introduce a layer-wise hybrid attention architecture by interleaving Sliding-Window Attention (SWA) with DeepSeek Sparse Attention (DSA) to boost long-context inference efficiency. By integrating Manifold-Constrained Hyper-Connections (mHC) with the Muon optimizer and targeted stabilization techniques, the model achieves faster training convergence without loss spikes during pre-training. At the system level, custom fused kernels and topology-aware co-optimizations eliminate critical memory, compute, and network bottlenecks, ensuring stable pre-training for massive MoE architectures scaling to a 512K context window. For post-training, we synergize parallel expert reinforcement learning with a strict training-inference alignment methodology. Finally, our dedicated inference engine leverages custom operator optimizations and parallelism strategies, alongside optimized cache management and asynchronous scheduling, to unlock exceptional generation throughput and ultra-low latency on Ascend hardware. We open-source our complete training and inference frameworks, alongside natively trained model checkpoints, at <https://gitcode.com/ascend-tribe>.

1 Introduction

Recent advancements in large language models (LLMs) have catalyzed a fundamental paradigm shift, transitioning these systems from passive conversational assistants into proactive, autonomous agents capable of navigating complex environments. This evolution demands that models reliably execute long-context planning, invoke external tools with precision, and maintain cognitive coherence across extended execution trajectories. However, modern deployment faces critical bottlenecks: generic architectures incur prohibitive inference costs and suffer severe performance degradation when scaling context windows, while standard alignment techniques fail to prevent temporal and structural reasoning errors during long-horizon execution. openPangu-2.0 is engineered from the ground up to address these challenges. By synergizing custom hardware kernels, system architecture, training-inference-aligned agentic RL, and dedicated inference acceleration, we establish a rigorous hardware-software co-design methodology, delivering an efficient, reliable, and cost-effective foundation purpose-built for long-context agentic tasks.

Model Architecture. To establish native long-sequence capabilities with high efficiency, we develop the core architectural advances across two architectural variants: openPangu-2.0-Pro and openPangu-2.0-Flash. We decouple context processing via a layer-wise hybrid attention mechanism, interleaving Sliding-Window Attention (SWA) [7] for dense local context with DeepSeek Sparse Attention (DSA) [14] for sparse global retrieval. For inference acceleration, we attach a three-head

Multi-Token Prediction (MTP) module [12, 22]. Furthermore, combining Manifold-Constrained Hyper-Connections (mHC) [58] with the Muon optimizer [12, 22] substantially boosts convergence speed under a fixed data budget. Finally, end-to-end training stability is guaranteed through targeted methods in MoE load balancing, mHC stabilization, MTP training strategies, and Muon parameter grouping, ensuring smooth, loss-spike-free pre-training.

Training Infrastructure. Scaling to a 512K context window while integrating complex architectural primitives (mHC, Muon, and MTP) poses severe communication and memory bottlenecks. To ensure long-context training stability, we establish a systematic co-optimization framework combining redundancy-free hybrid Context Parallelism (CP), lightweight point-to-point transfers for MTP, distributed overlap for Muon, and hybrid recomputation, heavily accelerated by custom AscendC fused kernels for mHC, parametric attention sinks (PAS), and ModAttn. To fully exploit the aggregate compute capacity of CloudMatrix 384 supernodes, we deploy a streamlined, topology-aware affinity strategy. Confining heavy traffic to supernodes and minimizing external transfers eliminates bottlenecks, maximizing hardware utilization.

Post-Training. Our post-training framework follows a three-stage pipeline: Supervised Fine-Tuning (SFT), Parallel Reinforcement Learning (RL) Expert Training, and Multi-Expert On-Policy Distillation (OPD) [1, 15, 21, 24, 59]. After a unified SFT stage, we train RL experts in parallel that separately optimizes domain-specialized policies (Reasoning, General, Agent, and Coding) over isolated data and reward to eliminate cross-domain interference in RL. Finally, an OPD training stage is adopted to fuse different RL experts. In addition to the current mechanisms that we employ to mitigate the training-inference mismatch issue, this report also includes a preview of a full-alignment solution on Ascend architecture to be adopted in our later model iterations.

Inference Acceleration. We engineer an Ascend-native inference framework that tightly couples hardware execution logic with runtime model dynamics. Our architecture introduces custom fused operators—including an inference-optimized mHC kernel and specialized collective communication primitives—alongside novel parallelism strategies such as Felix CP and per-core multi-stream execution. To maximize throughput, we incorporate tailored quantization strategies alongside an operator-aware, precision-preserving scheme, and integrate hybrid KV-cache management, Lossless Parallel Tokenization [45] (LoPT), and pioneering support for Automatic Prefix Caching (APC) and Multi-Token Prediction (MTP) tailored specifically for ModAttn. At a 128K context length, this framework unlocks exceptional long-context performance on Ascend NPUs: openPangu-2.0-Flash achieves an ultra-low latency of 5.63 ms TPOT and a generation throughput of 1,846 tokens/s per NPU (at 15 ms TPOT), while openPangu-2.0-Pro attains 9.55 ms ultra-low latency and 1,326 tokens/s per NPU (at 20 ms TPOT).

Performance and Applied Utility. Evaluation results demonstrate that openPangu-2.0 series achieves competitive performance across a variety of general, reasoning, and agentic benchmarks, while uncovering a critical synergistic feedback loop: strong foundational reasoning directly empowers complex agentic behaviors, which reciprocally sharpens multi-step planning and long-trajectory context processing. Moving beyond public benchmarks, we anchor openPangu-2.0 in practical, system-level utility. Taking smart devices as a primary case study, we aligned the model with the core capabilities required to drive the evolution of HarmonyOS toward a true AI-native Operating System (AIOS). To facilitate this evolution, we deliberately cultivated native support for intent-driven Generative User Interfaces (Generative UI) and execution-backed software synthesis. As showcased in real-world scenarios, openPangu-2.0 reliably transforms high-level human intent into production-ready, full-stack mini-apps and converts deep research workflows into dynamic, interactive knowledge artifacts.

2 Model Architecture

The openPangu-2.0 family comprises two Mixture-of-Experts (MoE) language models: openPangu-2.0-Pro (505B parameters, 18B active) and openPangu-2.0-Flash (92B parameters, 6B active). Both models natively support a 512K-token context window, with the overall architecture illustrated in Figure 1. To tackle the compute, memory capacity, and bandwidth bottlenecks inherent to long-context agentic workloads, we introduce a layer-wise hybrid attention mechanism that interleaves

DeepSeek Sparse Attention (DSA) with Sliding-Window Attention (SWA). This design elegantly decouples context processing: SWA layers specialize in capturing local dependencies, while DSA layers efficiently retrieve sparse global information. For highly efficient inference, we append three Multi-Token Prediction (MTP) layers to the backbone, substantially accelerating generation via self-speculative decoding. Furthermore, by integrating Manifold-Constrained Hyper-Connections (mHC) and optimizing via Muon, openPangu-2.0 achieves significant gains in learning efficiency under the same pre-training data budget. Comprehensive architectural specifications are detailed in Appendix A.1.

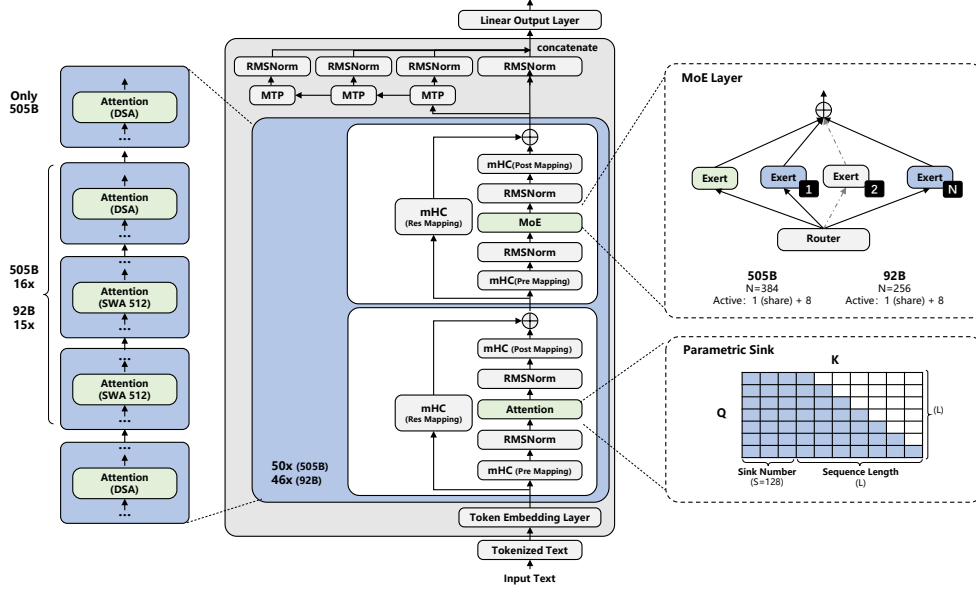


Figure 1: Overview of the openPangu-2.0 architecture. The first two layers of openPangu-2.0-Flash and the first three layers of openPangu-2.0-Pro are dense layers. An RMSNorm layer is applied to the residual streams at the first layer and every five layers thereafter.

2.1 Attention Architecture

Fundamentally built upon Multi-Head Latent Attention (MLA) [13], our attention modules employ an interleaved stack of DSA and SWA layers to drastically mitigate inference overhead in long-context regimes. To further refine the attention representations, we augment the standard MLA block with Parametric Attention Sinks (PAS) and Local-Context Modulated Attention (ModAttn)—the former systematically absorbs attention sink tokens, while the latter bolsters fine-grained local semantic modeling.

SWA. We adopt a repeated DSA-SWA-SWA pattern, corresponding to a 2:1 SWA-to-DSA layer ratio. In preliminary experiments, we observed that while larger ratios (e.g., 3:1 and 5:1) maintain performance on short sequences (4K), the training loss degrades when scaling up to 32K and 128K context lengths. The SWA window size is set to 512 tokens, and the RoPE [49] base frequency of SWA layers scales up with the context length, increasing from 1.0×10^4 at 4K to 6.4×10^6 at 512K. Although we explored alternative window sizes and RoPE base scaling strategies, they yielded marginal impacts on the final performance.

DSA. DSA introduces a lightweight Lightning Indexer (LI), which dynamically selects historical key-value (KV) entries relevant to the current token for attention computation, thereby eliminating massive redundant and irrelevant attention overhead. The selected subset is restricted to a fixed count of K tokens (e.g., 2,048), shifting the scaling factor of global attention costs from the full context length to the number of selected tokens.

At the late stage of the 512K context pre-training, we transition the model from the full-attention (FULL) architecture to the DSA architecture by replaying the 512K training data. Throughout this transition, an auxiliary learning objective is introduced for the LI to fit the native attention scores calculated by full attention, with the coefficient of this objective set to 0.1 (as a large coefficient yields excessive gradients that compromise training stability). After a brief dense warm-up and 100B tokens of sparse adaptation, the model mostly recovers in the downstream performance.

Unlike DeepSeek-V3.2, our base model is inherently a FULL-SWA hybrid. Thus, we only convert the full-attention layers to DSA, leaving the SWA layers intact. This structurally decouples context modeling: DSA extracts global dependencies, while SWA processes local interactions.

PAS. To mitigate the attention-sink phenomenon [23, 50, 56]—where semantically meaningless tokens absorb disproportionate attention mass—we augment each layer with parametric attention sinks (PAS). As illustrated in Figure 1, PAS introduces 128 learnable key-value pairs (latent KVs for MLA) per layer to absorb redundant attention. During inference, these act as constant-sized virtual KV entries, imposing negligible compute and cache overhead even in long-context scenarios. Empirically, PAS also accelerates training convergence for the hybrid SWA architecture. A detailed comparison with the sink mechanism in gpt-oss [38] is provided in Appendix A.2.

ModAttn. ModAttn addresses local ambiguity by injecting local context before and after the global attention computation using lightweight causal one-dimensional convolutions and a residual architecture. Formally, given a sequence representation $Z \in \mathbb{R}^{T \times d}$, the local modulation operator is defined as

$$\text{Mod}_\theta(Z) = Z + \text{CausalConv1D}_\theta(Z), \quad (1)$$

where θ denotes the parameters of the convolutional branch. ModAttn applies this modulation to the compressed latent branches (including both Q and KV branches) of MLA. Subsequently, a similar modulation operation is applied after the global aggregation by FlashAttention [11] but prior to the output projection. ModAttn thus provides a low-intrusion enhancement that moves local semantic conditioning ahead of the global attention computation without altering its interface.

Efficiency Analysis. The proposed DSA-SWA architecture lies between two representative alternatives: a homogeneous DSA stack (e.g., DeepSeek-V3.2) and an alternating FULL-SWA stack (e.g., Gemma 2) [51]. Rather than repeating the complete complexity expressions, Table 1 compares the leading sequence-length coefficients governing the dominant costs of long-context serving: prefill FLOPs, decode-time cache reads, and decode-time cache memory.

Table 1: Comparison of representative long-context attention architectures. Entries report the leading coefficients with respect to sequence length. FULL-SWA and the proposed DSA-SWA both adopt a 1:2 global-to-local layer ratio.²

Cost	Homogeneous DSA	FULL-SWA	DSA-SWA
Prefill FLOPs (S^2)	$N_{\text{head}}^{\text{Indexer}} D_{\text{Indexer}}$	$\frac{1}{3} N_{\text{head}} (D_{\text{QK}}^{\text{MHA}} + D_{\text{V}}^{\text{MHA}})$	$\frac{1}{3} N_{\text{head}}^{\text{Indexer}} D_{\text{Indexer}}$
Decode cache reads (S)	D_{Indexer}	$\frac{1}{3} D_{\text{KV}}$	$\frac{1}{3} D_{\text{Indexer}}$
Decode cache memory (S)	$D_{\text{KV}} + D_{\text{Indexer}}$	$\frac{1}{3} D_{\text{KV}}$	$\frac{1}{3} (D_{\text{KV}} + D_{\text{Indexer}})$

Compared with a homogeneous DSA stack, the proposed architecture confines DSA to one third of the layers, proportionally reducing the cost of global token selection during both prefill and decoding. Compared with an alternating FULL-SWA stack, it replaces dense global attention with lightweight indexing followed by sparse retrieval, reducing the dominant quadratic computation from the full-attention representation to the much smaller index representation while simultaneously lowering decode-time cache reads. Consequently, the proposed DSA-SWA architecture achieves a more favorable balance among prefill FLOPs, decode-time cache reads, and decode-time cache memory, making it well suited for long-context serving.

¹ N_{head} and $N_{\text{head}}^{\text{Indexer}}$ denote the numbers of attention heads and indexer heads, respectively. $D_{\text{QK}}^{\text{MHA}}$, $D_{\text{V}}^{\text{MHA}}$, D_{KV} , and D_{Indexer} denote the per-head query/key dimension, per-head value dimension, per-token cache width, and indexer dimension, respectively.

2.2 Mixture of Experts

openPangu-2.0 scales up the number of experts (with the Pro version featuring 384 experts) and adopts a high sparsity ratio (activating 8 out of 384 routed experts), which places more stringent demands on load balancing. To maintain balanced expert workloads during training, we implement two complementary mechanisms: an adaptive auxiliary-loss-free load balancing strategy to manage balancing across the entire global batch, and an expert-parallel group (EP-group) auxiliary loss to control local balancing within each EP-group.

Adaptive Auxiliary-Loss-Free Bias. Auxiliary-loss-free balancing [53] adjusts a per-expert selection bias b_i according to the difference between the observed load F_i and the target load $Q_i = 1/N_r$, where F_i is the fraction of tokens routed to expert i over the global batch. The original formulation updates the bias with a sign function, which applies a correction of equal magnitude regardless of the degree of overload. Instead, we use an RMS-normalized update [66] as shown in Equation 2:

$$b_i \leftarrow b_i - \gamma \cdot \frac{F_i - Q_i}{\text{RMS}(F - Q)}, \quad \text{RMS}(F - Q) = \sqrt{\frac{1}{N_r} \sum_{j=1}^{N_r} (F_j - Q_j)^2}. \quad (2)$$

This update preserves the relative magnitude of the expert load errors, so experts with larger overload receive larger corrections and experts close to balance receive smaller ones, while the overall update scale remains controlled by the learning rate γ . Our preliminary findings indicate that the adaptive method achieves stronger load control than the non-adaptive baseline while maintaining the same convergence rate. For a detailed comparison with empirical results, please refer to Appendix A.3.

Expert-Parallel-Group Auxiliary Loss. A sequence-level auxiliary loss constrains expert load within each individual sequence, which can over-constrain routing. Instead, the balancing statistics are aggregated over the tokens within an EP-group. With f_i denoting the fraction of tokens routed to expert i and p_i the average router probability for expert i , the loss is defined as Equation 3:

$$\mathcal{L}_{\text{ep}} = \alpha_{\text{ep}} \cdot N_r \sum_{i=1}^{N_r} f_i p_i. \quad (3)$$

Since the expert load within an EP-group directly determines its computation and communication cost, this loss (scaled by hyperparameter α_{ep}) maintains expert-parallel efficiency while imposing a looser local constraint than a sequence-level loss.

2.3 Manifold-Constrained Hyper-Connection

Formulation. Although the standard residual connection $x + f(x)$ effectively mitigates the problem of gradient vanishing in deep networks, this simple additive combination tends to reduce the diversity of the information flow within the backbone, leading to highly similar representations across deep layers. Building on Hyper-Connections [65], mHC addresses this limitation by replacing the single residual stream with n parallel residual streams.

For each sub-layer, mHC generates three sets of connection weights from the current multi-stream state $\bar{h}$:

$$\bar{x} = \text{RMSNorm}(\bar{h}), \quad [r_{\text{pre}}, r_{\text{post}}, r_{\text{res}}] = \bar{x} W_{\text{mHC}}, \quad (4)$$

$$H_{\text{pre}} = \sigma(r_{\text{pre}}), \quad H_{\text{post}} = 2\sigma(r_{\text{post}}), \quad H_{\text{res}} = \text{Sinkhorn-Knopp}(\text{reshape}(r_{\text{res}}, n, n)). \quad (5)$$

where σ denotes the sigmoid function. Here H_{pre} selects which streams are read into the sub-layer input, H_{post} determines how the sub-layer output is written back to the streams, and H_{res} controls the cross-stream mixing along the residual path. mHC applies a manifold projection to H_{res} , constraining it to be approximately doubly stochastic.

Normalization for Stable Training. Under the original mHC placement, where H_{pre} is applied before pre-normalization and H_{post} after the sub-layer output, we observe a gate-collapse phenomenon: during training, all H_{pre} and H_{post} gates across the entire layer approach zero for early layers. Consequently, these layers decouple from the residual streams, degrading the training signal. We hypothesize that this collapse stems from the unstable scale of the sub-layer output prior to write-back.

openPangu-2.0 stabilizes mHC through a two-level scale control mechanism. First, sandwich normalization [18] is applied to the sub-layer output before the H_{post} write-back, relieving H_{post} from suppressing large or unstable output scales. Second, block-level post-normalization is applied to the residual streams after the first layer and every five layers thereafter, explicitly bounding the accumulated scale of the multi-stream trunk across depth. These modifications successfully mitigate the gate-collapse phenomenon and we attain the performance gains reported in the original paper. As demonstrated in Table 2, the stabilized mHC consistently outperforms the baseline across downstream tasks.

Table 2: Downstream evaluation of mHC and the baseline (MoE15B-A2B) after 1T training tokens. The baseline MoE15B-A2B adopts the standard openPangu-2.0 architecture but without the mHC module (for detailed architecture hyperparameters, see Appendix A.1). The *EN* and *ZH* columns are multi-task few-shot evaluation suites measuring the fundamental English and Chinese capabilities, respectively, covering general NLP tasks such as reading comprehension, knowledge question answering, etc.

Model	EN	ZH	MMLU	CEval	BigBench	MATH	MBPP+	Average
Baseline	0.500	0.605	0.765	0.708	0.591	0.459	0.532	0.594
mHC	0.511	0.608	0.799	0.743	0.622	0.472	0.553	0.615

2.4 Multi-Token Prediction

openPangu-2.0 adopts MTP for speculative decoding. The main model carries auxiliary layers that predict several future tokens from the current hidden state. During inference, these layers can propose draft tokens for speculative decoding, improving hardware utilization when decoding is memory-bandwidth bound. Alongside the main backbone output, MTP introduces D prediction layers to predict D future tokens, respectively. The total training loss is defined as

$$\mathcal{L} = \mathcal{L}_{\text{main}} + \sum_{k=1}^D \lambda_k \mathcal{L}_{\text{MTP}}^{(k)}, \quad (6)$$

where $\mathcal{L}_{\text{main}}$ denotes the cross-entropy loss of the main backbone model, and D is the number of MTP layers. $\mathcal{L}_{\text{MTP}}^{(k)}$ represents the cross-entropy loss of the k -th MTP layer predicting the k -th future token, and λ_k is the loss coefficient for the k -th MTP layer, which is used to balance the contribution of each head to the total loss.

Specifically, the deployed architecture set $D = 3$. This provides the self-speculative decoder with a multi-token draft path while keeping the main autoregressive model unchanged. To keep the auxiliary module practical for long contexts, all attention inside the MTP module uses sliding-window attention with a fixed window of 2048. The window bounds compute and cache growth while preserving enough local context for high-quality draft predictions.

Training Strategy. openPangu-2.0 uses a stage-adaptive MTP schedule. The early pretraining stage uses one MTP layer with a relatively larger coefficient of 0.3, so that the model receives denser future-token supervision while the representation is still forming. During reasoning phase, the coefficient is reduced to 0.1 to avoid over-regularizing the main reasoning objective. In the long-context extension stage, the MTP module is expanded to three layers and the coefficient is retained at 0.1 for each layer (totaling 0.3), producing the final self-speculative draft layer used for inference.

Table 3: MTP layer count and loss weight by training stage.

Phases	Sequence length	MTP layers D	Loss weight λ
General Phase	4K	1	0.3
Reasoning Phase	8K	1	0.1
Annealing Phase	32K, 128K, 512K	3	0.3

2.5 Muon Optimizer

openPangu-2.0 applies the Muon optimizer to all matrix-shaped parameters. Muon orthogonalizes the update matrix via Newton-Schulz iteration, maintaining an approximately uniform spectral norm for the update steps. This prevents the suppression of rare gradient directions, aiding the acquisition of long-tail knowledge. Additionally, because Muon tracks only first-order momentum, it halves the optimizer state memory compared to AdamW.

In practice, Muon optimizes all two-dimensional and higher-dimensional weight tensors, while AdamW optimizes word embeddings, the prediction head, parametric sink parameters, and one-dimensional parameters (e.g., RMSNorm weights and mHC gate coefficients). For the MLA query and key-value up-projections, attention heads are partitioned into four groups before applying Muon. This grouping aligns with the structural independence of the heads while keeping the update matrices sufficiently large for stable orthogonalization. The Newton-Schulz iteration uses a fixed number of steps, with customized per-iteration coefficients to better approximate the orthogonal factor (see Appendix A.4).

Previous work [34] reported that Muon can cause attention logits (i.e., $QK^\top$) to explode, leading to training instability. However, the attention logits in openPangu-2.0 remain stable throughout pre-training. We attribute this stability to the architectural designs of sandwich normalization and block post-normalization, which explicitly constrain the output scale. Further empirical details are provided in Appendix A.5.

2.6 Tokenizer

The openPangu-1.0 tokenizer follows a conventional BPE vocabulary design based on token co-occurrence frequency. Although effective for general text, it provides inconsistent segmentation for numbers and relatively low compression on multilingual and industry corpora. This limitation is particularly relevant to multimodal and agentic tasks, where numbers frequently represent coordinates, bounding boxes, indices, and tool parameters.

openPangu-2.0 retrains the tokenizer using a rebalanced mixture of Chinese, English, code, industry documents, and 188 low-resource languages. We also redesign numerical pre-tokenization and evaluate two variants. The *3-digit* variant preserves moderate numerical compression, whereas the *single-digit* variant splits contiguous Arabic numerals into individual digits to provide a uniform numerical representation.

We evaluate tokenizer efficiency using the average number of characters covered per token, where a higher value indicates better compression. The results are reported in Table 4.

Table 4: Vocabulary compression rate comparison.

Vocabulary	Vocab Size	Chinese	English	Multilingual	Code	Industry	Average
openPangu-1.0	153,376	3.82	4.30	2.80	2.80	4.18	2.56
openPangu-2.0-3digit	150,400	4.04	4.27	3.37	2.77	4.51	2.71
openPangu-2.0-single-digit	149,600	3.82	4.15	3.32	2.56	4.27	2.59

The 3-digit variant achieves the best overall compression, especially on Chinese, multilingual, and industry corpora. Nevertheless, openPangu-2.0 adopts the single-digit variant as its final tokenizer. Although this design introduces a small compression penalty, it avoids frequency-dependent merging of multi-digit numbers and provides more regular digit-level representations for numerical reasoning, multimodal localization, and tool-use scenarios.

2.7 On-Device Model

The model features a 30B-parameter MoE architecture with only 2B active parameters, enabling efficient on-device deployment. It adopts Kirin-aware quantization and pruning to reduce computation and memory overhead, achieving a 50% inference speedup and a 20% reduction in memory footprint. To alleviate the frequent expert swapping caused by sparse MoE routing on resource-constrained devices, an expert reuse loss encourages adjacent tokens to activate similar sets of experts, reducing the expert-switching frequency by 50%. In addition, expert activation prediction anticipates the experts likely to be selected by subsequent tokens and enables their proactive prefetching, thereby reducing

expert-loading stalls and improving model throughput by up to $5\times$. Together, these optimizations align the model’s computation, memory access, and expert scheduling with the characteristics of Kirin hardware, making the model highly efficient for on-device inference.

3 Pre-training

3.1 Data Construction

The openPangu-2.0 pre-training corpus comprises roughly 34 trillion high-quality, diverse tokens, representing a significant upgrade over the openPangu-1.0 models. We completely refined our data construction pipelines to aggressively scale the corpus size and extend the context length. To optimize learning, the pre-training process is structured into a three-stage curriculum. Each phase utilizes tailored data recipes and selection strategies to progressively develop the model’s capabilities:

Phase 1: General (25T tokens): Focuses on establishing a robust foundation of general knowledge and broad linguistic capabilities.

Phase 2: Reasoning (8T tokens): Targets the enhancement of deep reasoning, logic, and advanced coding skills.

Phase 3: Annealing (1.4T tokens): Refines model behavior and agentic abilities. We intentionally reserve long documents and complex trajectory data for this final phase to maximize the model’s capacity for autonomous tasks.

Our pre-training corpus aggregates a wide variety of sources, including web pages, books, PDFs, multilingual text, code repositories, STEM (Science, Technology, Engineering and Mathematics) literature, industrial domain data, agentic tasks, and synthetically generated data. Building upon the openPangu-1.0 foundation, we heavily prioritized scaling the diversity and quality of ARC (Agent, Reasoning, and Code) data. We accomplished this by overhauling our rule-based heuristics and integrating robust, model-based evaluation pipelines. Furthermore, by leveraging diverse evaluation harnesses, we generated and refined high-fidelity synthetic agent trajectories specifically to empower openPangu-2.0’s agentic capabilities.

To preserve valuable long-tail knowledge from data samples with low cleanliness or fluency scores, we developed a few specialized, fine-tuned models. These models rewrite and standardize the text format while strictly preserving the original semantic meaning. Furthermore, every document in the corpus is annotated with rich metadata, including: quality and difficulty scores; topical categories; educational value. These detailed labels drive a highly curated, curriculum-based sampling strategy, ensuring the model is exposed to the optimal distribution of data at every phase of its training.

3.2 Training Details

Both the openPangu-2.0-Flash and openPangu-2.0-Pro models share the same training curriculum, auxiliary loss designs, and optimizer configurations, with differences only in batch size and learning rate schedules.

Training Curriculum. We pre-train the models using a three-stage curriculum. In the first *general* stage, we train at a 4K sequence length on 25T tokens. For the Flash model, we gradually warm up the batch size to 36M and the learning rate to a peak of 3.0×10^{-4} over the first 500B tokens. We maintain this peak learning rate for the next 17T tokens, followed by a cosine decay down to 1.0×10^{-4} over the final 8T tokens. For the Pro model, we use the same token schedule but warm up the batch size to 64M and use a peak learning rate of 2.0×10^{-4} instead. In the second *reasoning* stage, we increase the proportion of high-quality STEM and code data, continuing training on 8T tokens at an 8K sequence length, while applying a cosine decay to the learning rate from 1.0×10^{-4} to 3.0×10^{-5} . The third *annealing* stage extends the context length successively to 32K, 128K, and 512K using 800B, 400B, and 200B tokens, respectively, and cosine-anneals the learning rate from 3.0×10^{-5} down to 8.0×10^{-6} . After the main curriculum, we introduce sparse attention through DSA, with a short dense warm-up on 8B replay tokens at a learning rate of 1.0×10^{-3} , followed by sparse training on 100B replay tokens at a learning rate of 8.0×10^{-6} .

Auxiliary Configurations. For MoE load balancing, we apply adaptive auxiliary-loss-free balancing (with an update learning rate of 1.0×10^{-3}), alongside the EP-group auxiliary loss and router z-loss with weights of 2.0×10^{-4} and 1.0×10^{-6} , respectively. These balancing constraints are applied exclusively during the general stage and disabled afterwards. For MTP, we use 1 layer with a loss weight of 0.3 in the general stage, reduce the weight to 0.1 in the reasoning stage, and expand to 3 layers in the annealing stage. Across stages, we gradually scale the RoPE base with the context length, setting the RoPE base to 10K at the 4K sequence length, 100K at 8K, 400K at 32K, 1.6M at 128K, and 6.4M at 512K. For mHC, we configure 4 parallel residual streams and apply 20 Sinkhorn-Knopp iterations for the manifold projection.

Optimizer Configurations. We employ Muon alongside AdamW for different parameter groups. For AdamW, we set $\beta_1 = 0.9$, $\beta_2 = 0.95$, and $\epsilon = 1.0 \times 10^{-8}$. For Muon, we set the momentum $\mu = 0.95$ and apply Nesterov acceleration. We scale its update by a factor of $\gamma = 0.2$ to match the RMS magnitude of AdamW, thereby allowing both optimizers to share a unified learning-rate schedule. We apply a global weight decay of $\lambda = 0.1$ to all parameters except word embedding and RMSNorm weights.

3.3 Training Infrastructure

While openPangu-2.0 builds upon the training infrastructure of openPangu-1.0 [62], its upgraded architecture introduces a new class of system challenges. Scaling to a 512K context window—combined with novel components like mHC, Muon, MTP, and ModAttn—resulted in highly complex distributed communications, severe activation-memory pressure, and a lack of efficient Ascend-native implementations for the newly introduced operators.

To break these bottlenecks, we engineered a comprehensive optimization framework focused on three key areas:

Communication Overlap and Efficiency: Our system eliminates redundancy in hybrid CP communication, utilizes a lightweight P2P protocol for MTP, and introduces a distributed computation-communication overlap for Muon.

Supernode-Aware Parallelism: Our parallel configuration strategy leverages CloudMatrix 384 supernode affinity grouping. The core design principle is to confine heavy traffic—Tensor Parallelism (TP), CP and Expert Parallelism (EP)—strictly within the supernode to exploit its high-bandwidth super-plane.

Algorithmic and Hardware Co-Design: We developed custom AscendC fused kernels for mHC, PAS, and ModAttn to provide native hardware acceleration. To alleviate memory pressure, we implemented intra- and inter-layer hybrid recomputation specifically tailored for mHC.

By insulating heavy intra-supernode communications from external links and overlapping parameter-network traffic with computation, this co-designed architecture eliminates inter-node bottlenecks and maximizes hardware throughput.

3.3.1 Ascend-Optimized mHC Training

mHC introduces real overhead: increased memory footprint, heavy inter-stage traffic, and throughput bottlenecks in vector processors during dense tensor operations. We tackle these issues with targeted optimizations in the sections ahead.

mHC Intra-layer / Inter-layer Recomputation. To overcome the linear growth of mHC activation memory that limits batch size and long-sequence training, we propose two fine-grained selective recomputation schemes. The intra-layer scheme caches only three tensors (backbone input to attention, attention output, and multilayer perceptron (MLP)/MoE output) and hides recomputation cost by overlapping it with intra-layer communication, targeting setups without pipeline parallelism (PP). For PP with virtual pipeline parallelism (VPP), the cross-layer scheme caches just the first layer’s attention input per chunk and recursively recovers the rest during communication idle slots, adding no extra FLOPs and leaving iteration time unaffected. Both schemes share the same core mechanism; cross-layer only adjusts the recomputation timing for PP, allowing flexible choice based on resources and objectives.

PP Communication Optimization under mHC. In PP scenarios, mHC and multi-head MTP both amplify hidden states—mHC by stream count, MTP by prediction heads. To address this issue, we send only main-branch states across PP boundaries, and expand MTP inputs locally at the target stage. This will be detailed in the Load Balancing for Multi-Token Prediction Training section 3.3.4.

Ascend-Affinity mHC Fused Operators. We implement H_{pre} , H_{post} and H_{res} with AscendC to maximize performance, particularly optimizing vector computations. The implementation-level optimizations are described in the following.

To prevent hardware idling due to serial execution of stages, we apply a prefetching strategy that eliminates stalls in the Vector pipeline caused by waiting for CUBE results, achieving deep overlap between Vector and CUBE pipelines. Combined with multi-level hierarchical fine-grained parallelism, minimized workspace data movement to boost L2 cache hit rates, and prefetching overlapped with computation, we effectively hide memory latency and deliver peak performance for individual operators.

Based on a joint analysis of memory-bound and compute-bound characteristics, we use the matrix dimension n as the threshold to dynamically switch computation between Vector and CUBE cores. This offloading balances the bottlenecks between the two units and maximizes hardware utilization. For low-level instruction tuning, we adopt the standard *Muls+Axpy* instruction sequence on the Vector cores to implement high-precision FP32 matrix multiplication, fully exploiting vector core throughput without sacrificing accuracy.

3.3.2 512K Context Training

CP is essential for scaling Transformer training to ultra-long sequences. While Ulysses CP [28] is highly efficient, its head-wise partitioning requires the number of attention heads to be divisible by the CP degree. More critically, it interacts poorly with the sparse flash attention (SFA) operator on Ascend NPUs, limiting its viability for DSA training workloads. To address this, we adopt key-value context parallelism (KV CP) as our default strategy in sparse-attention training stage. By sharding the sequence dimension instead of the heads, KV CP maintains full compatibility with the underlying sparse-attention kernels. For full-attention training, we introduce Hybrid CP, which combines Ulysses and KV CP to bypass head divisibility constraints while significantly reducing communication volume and peak KV buffer size compared to pure KV CP.

While this framework successfully enables stable training at the 512K scale, pushing the context window this far exposes two critical bottlenecks. Specifically, we must address the redundant KV communication caused by sequence parallelism, as well as the challenge of efficiently incorporating PAS into DSA training. The following sections describe our targeted optimizations for both issues.

KV CP Communication Redundancy Optimization. Standard KV CP gathers global KV tokens from all ranks along the sequence dimension. However, each rank only needs a specific subset of these tokens for its local queries. This introduces substantial redundant communication in long-context training, particularly when a global training sample packs multiple short documents. To eliminate this redundancy, we specialize the communication optimization for SWA, full attention (FA), and DSA layers:

In **SWA** layers, the query tensor on a given rank only needs to attend to the last window-size KV tensors from the preceding rank. We replace the global AllGather communication with targeted P2P communication. Each rank now only receives the necessary window-size tensors from the previous rank and passes its own to the next. This drastically reduces the communication volume from $S \cdot B \cdot H$ to $W \cdot B \cdot H$, where S is the complete sequence length, W is the sliding-window size, B is the batch size, and H is the hidden dimension.

In **FA** and **DSA** layers, queries only require KV tensors from the same sub-sequence that reside on previous ranks, rather than KV tensors from all ranks. Thus, we replace AllGather with AllToAllV to eliminate this redundancy, as shown in Figure 2. This ensures precise, on-demand data transfer, as each rank only sends and receives the exact KV segments required by the sub-sequence.

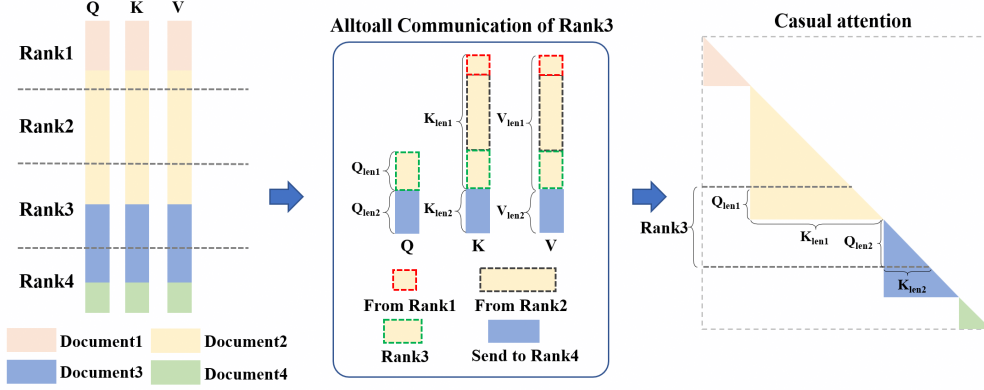


Figure 2: AllToAllV communication when CP=4.

FA Rescale. PAS are introduced as trainable key-value parameters that are conceptually prepended to every single sequence as persistent attention anchors. Although the fused PAS operator is more efficient when supported, it lacks the generality required for diverse training environments. To bridge this gap, we introduce FA Rescale as a flexible fallback. This method performs two standard attention calls and reconstructs the result of a single attention over the concatenated main sequence and sink tokens.

Let A and B represent the logit partitions for the main sequence and the parametric attention sinks, respectively, and let $O(X)$ denote the attention output for logit partition X . FA Rescale processes these branches independently and merges them as follows:

$$O(A+B) = O(A)\alpha_A + O(B)\alpha_B. \quad (7)$$

The rescale coefficient for branch $X \in \{A, B\}$ is computed as

$$\alpha_X = \frac{l_X \exp(m_X - m^*)}{l_A \exp(m_A - m^*) + l_B \exp(m_B - m^*)}, \quad (8)$$

where (m_X, l_X) are the softmax-max and softmax-sum statistics returned by each FA call. Here $m^* = \max(m_A, m_B)$ is introduced for numerical stability. This is mathematically equivalent to a single global softmax over $A+B$, completely bypassing the need for native sink support in the FA kernel. By maintaining the rescaling factors in FP32 precision, the transformation is entirely lossless in the FP32 regime, subject only to negligible bfloat16 (BF16) rounding artifacts.

FA Rescale is highly flexible in deployment, as it operates entirely on top of standard FA calls and incurs neither extra memory nor measurable computational overhead. This flexibility is particularly valuable for DSA training, which relies on clean, sink-free sparse-attention statistics to compute a Kullback-Leibler (KL) divergence loss. FA Rescale naturally preserves this isolation by applying the SFA operator strictly to the main sequence while handling the sink branch with a lightweight FA call. Merging these outputs via Eqs. (7) and (8) yields a complete output that incorporates the parametric attention sinks. This enables efficient DSA training without contaminating the KL loss or requiring a redundant, sink-free SFA pass.

3.3.3 Distributed Muon Optimizer

The Muon optimizer achieves symmetry and curvature adaptation through Newton-Schulz (NS) orthogonalization of the momentum matrix. Compared to elementwise optimizers, it faces three major challenges: NS computation depends on complete weight matrices, incurring significant NS-input aggregation communication overhead; NS iterations are constrained by host-side scheduling, leading to prominent scheduling overhead; and cross-rank redundant weight computation is severe. To address these issues, we propose a communication-computation joint optimization scheme for the Muon optimizer based on a distributed data parallel (DDP) bucket implementation, referred to as HyperParallel Muon+.

NS-input Aggregation Communication Optimization. To reduce the aggregation overhead, we propose two aggregation strategies —AllGather and AllToAll —and implement them for the DDP bucket Zero Redundancy Optimizer (ZeRO) scenario [43].

In the DDP bucket ZeRO scenario, weights are distributed across buckets, and ZeRO partitioning uniformly divides each bucket. Only the weight data at partition boundaries requires communication. AllGather, as the baseline scheme, aggregates the entire bucket so that each rank holds the complete bucket weights and can extract the required parameters for NS iteration. Each rank processes only the weights within its local partition, while boundary weights are redundantly computed across multiple ranks. However, when a bucket is large and contains multiple complete parameters with only boundary parameters being incomplete (as opposed to small buckets where a single parameter spans multiple buckets), AllGather incurs communication redundancy. In this case, the AllToAll scheme is adopted: boundary parameters are first extracted, excluding those already complete locally; adjacent-rank exchange is then performed on the boundary parameters, i.e., AllToAllV semantics. Note that AllToAllV is a P2P communication primitive that incurs substantial queue buffer resource overhead across supernodes, and therefore should not be used without restriction in large-scale clusters. Accordingly, to address the cross-plane queue overflow issue of AllToAllV, we draw on the two-layer communication design of DeepEP [12]: the data-parallel (DP) domain is decomposed into a $DP_{\text{outer}} \times DP_{\text{inner}}$ two-dimensional topology, where DP_{outer} and DP_{inner} denote the outer and inner dimensions, respectively. The outer dimension performs symmetric AllGather communication for cross-hyperplane scaling, and the inner dimension executes fine-grained AllToAllV exchange that remains controllable within the domain.

Bucket-level Communication-Computation Masking Optimization. To hide the host-side scheduling overhead, bucket-level communication-computation masking is implemented. Asynchronous communication of the current bucket runs in the background, while momentum/update-matrix orthogonalization and parameter update are carried out on the previous bucket, with the two operations overlapping on the timeline. The timing of the masking pipeline is illustrated in Figure 3: the communication operation of Bucket i overlaps with the computation operation of Bucket $i - 1$, where i indexes the buckets, while the communication of the first bucket and the computation of the last bucket constitute unmaskable exposed overhead. When communication latency is less than or equal to computation latency, communication overhead can be fully masked, and end-to-end runtime is determined solely by pure computation. The masking benefit depends on the relative relationship among bucket granularity, network bandwidth, and computation density. The actual gain follows the same rule: communication can be sufficiently masked in small-scale clusters, whereas communication typically becomes the bottleneck in large-scale clusters.

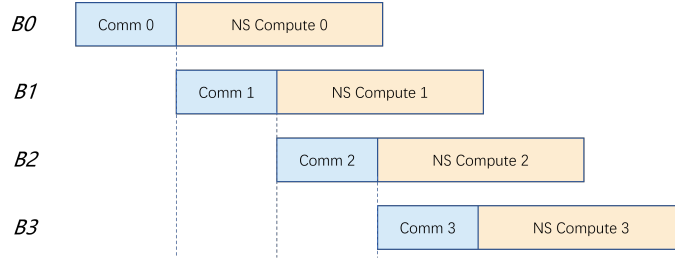


Figure 3: Muon communication and compute overlapping.

3.3.4 Load Balancing for Multi-Token Prediction Training

MTP introduces additional computation, memory consumption, and PP communication. These costs become particularly significant when MTP is combined with mHC, which replace the single residual stream with multiple parallel residual streams. A naive implementation transfers MTP states proportional to $n \times D$ across PP boundaries, where n and D retain the residual-stream and prediction-head meanings defined above, while placing all MTP layers, the output projection, and the loss computation at the end of the pipeline creates a severe tail-stage bottleneck.

openPangu-2.0 addresses these issues through three complementary optimizations. First, normal PP boundaries transfer only the mHC-expanded backbone hidden states and their gradients. MTP auxiliary states are generated locally on the stage containing the first MTP layer, and their gradients remain local to the stages where they are consumed. For three-head MTP, this reduces the effective PP payload from four streams—one backbone stream and three prediction streams—to one backbone stream, reducing MTP-related P2P communication by 75%. With two mHC residual streams

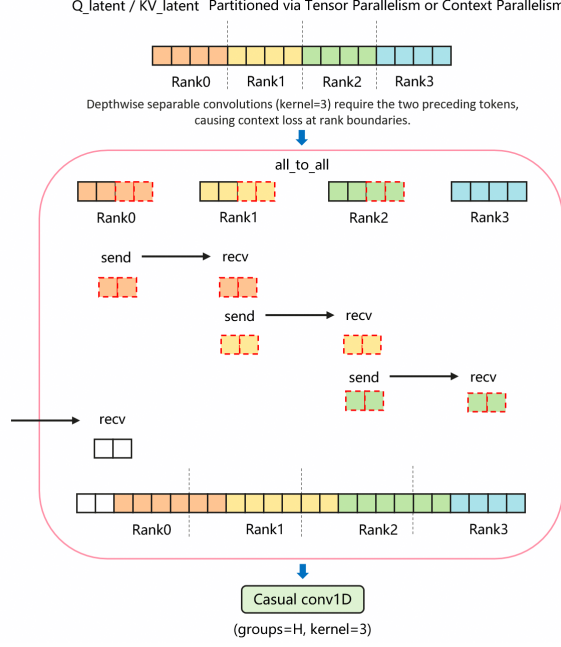


Figure 4: ModAttn Zero-redundancy neighborhood communication.

and three MTP heads, it also avoids the approximately $6\times$ communication amplification of the naive design.

Second, MTP layers participate in the PP/VPP layout rather than being concentrated at the final stage, allowing their computation to be distributed across pipeline stages. Third, the output projection and loss computation on the final stage are divided into sequence chunks. Each chunk performs forward computation followed immediately by backward computation, after which its logits can be released. With C chunks, the theoretical peak logits residency can approach $1/C$ of that required for the full sequence, subject to buffering and scheduling overheads.

Together, backbone-only P2P transfer, MTP-aware pipeline partitioning, and chunk-level immediate backward separately mitigate the communication, computation, and memory bottlenecks of MTP training. In the openPangu-2.0 training configuration, the optimized layout improves training throughput by approximately 20% compared with the unbalanced MTP layout.

3.3.5 System-Level Optimization for ModAttn

ModAttn enhances openPangu-2.0’s MLA using sliding-window convolutions and masking. To resolve deployment challenges in ultra-long sequences—cross-node boundary precision loss, activation memory explosion, and data movement bottlenecks—we restructured the system through three key optimizations.

Zero-redundancy neighborhood communication. Sequence parallelism truncates boundary information in ModAttn’s depthwise convolutions. To acquire cross-device context efficiently, we replace complex P2P or redundant AllGather methods with an All-to-All communication mechanism. Each computing node extracts its right-boundary activation and routes it to the adjacent downstream node. The receiving node concatenates this data during the forward pass, restoring the full receptive field as shown in Figure 4. This guarantees single-card equivalence without precision loss.

Zero-redundancy activation recomputation. To reduce memory pressure, we discard the padded FlashAttention [11] output (pad_attention) immediately after the forward pass rather than caching it. During backward propagation, the system dynamically re-executes zero-padding using the retained attention_out. Under BF16 precision, this strategy saves $\frac{B \times S \times H}{TP \times CP} \times 2$ bytes of activation memory per layer (where B, S, H denote batch size, sequence length, and hidden di-

mension; TP and CP represent tensor- and context-parallel degrees). This significantly mitigates out-of-memory (OOM) risks with negligible computational overhead.

Customized operator fusion. Since 1D causal convolution is inherently memory-bound, traditional NPU Cube unit execution suffers from massive data transpositions, achieving only 5% effective computation time. Instead, we directly utilize Vector units for element-wise multiply-accumulate operations, preserving the original data layout. Combined with a Rolling State Machine (RSM) that pipelines token processing via on-chip registers (y_0, y_1, y_2) , where y_0, y_1, y_2 denote the three register-resident rolling states, we fundamentally eliminate data movement overheads, drastically improve cache utilization, and reduce the module’s computation time by 79%.

4 Post-training

4.1 Supervised Fine-Tuning (SFT)

During supervised fine-tuning, we use a context length of 512K tokens to preserve long-context capability. The SFT dataset of openPangu-2.0 is built on a **dual-mode fusion strategy**, seamlessly integrating both *thinking-mode* and *non-thinking-mode* data. To ensure high-quality supervision, we implement a domain-specialized construction strategy across four core pillars:

Data Domain	Construction Strategy & Characteristics
Reasoning	Prioritizes complete chain-of-thought and interpretability. Every sample includes explicit intermediate steps, logical justifications, and strict result verification.
General Chat	Curated using a multi-dimensional evaluation rubric to guarantee responses are logical, concise, and highly aligned with human preferences.
Coding	Sourced from real-world Issue/Pull-Request pairs and rigorously filtered. We utilize a large-scale suite of verifiable execution environments across multiple languages, with a special emphasis on AscendC and ArkTS for HarmonyOS development.
Long-Trajectory Agent	Generated via a multi-stage pipeline where the model executes complex tasks across real HarmonyOS and simulated sandbox environments using different agent harnesses. It broadly covers a wide range of tool use, skill use, file operations, and coding capabilities.

Quality Control: Trajectories containing erroneous steps are identified via environment feedback and a reward model. We mask these incorrect transitions during training, forcing the model to learn exclusively from high-quality, correct actions.

4.2 Parallel Reinforcement Learning (RL) Expert Training

Due to heterogeneous task structures and conflicting feedback signals, applying reinforcement learning to general-purpose large language models (LLMs) is a challenging task. For example, coding tasks have binary, verifiable outcomes, whereas general conversations are open-ended and subjective. Optimizing these competing objectives in a single RL stage often leads to domain interference and signal imbalance. To address this, we utilize a *parallel expert-training strategy* to separately train domain-specialized policies. This separates domain-specific policy optimization, enabling independent expert updates while reducing cross-domain interference. All expert policies are optimized using Group Relative Policy Optimization (GRPO) [46] without a KL regularization term.

4.2.1 Reasoning RL

Reasoning RL benefits from tasks that are verifiable, appropriately challenging, and diverse. To support this, we cover four task categories: math, science, logic games, and tool-integrated reasoning. We construct a candidate data pool using the following curation steps:

- **Quality Filter:** Uses an LLM evaluator to score query formulation, discarding sub-par prompts, multi-part sub-questions, and ambiguous answers.
- **Diversity Balancing:** Annotates remaining queries with multi-level subject labels and fine-grained knowledge points to ensure balanced representation.

For tool-integrated reasoning (e.g., asynchronous Jupyter environments), we additionally apply penalties for Python parsing errors during interleaved thinking and mask out earlier correct rounds so that only the final error is penalized.

4.2.2 General RL

Designed to handle diverse, user-facing tasks, General RL focuses on open-ended queries where multiple valid responses exist. We target four core capabilities:

- **Instruction Alignment:** Strict adherence to explicit and implicit constraints within system prompts and contexts.
- **Factual Reliability:** Minimizing hallucinations, rejecting unsupported claims, and curbing excessive abstention.
- **Long-Context Understanding:** Locating, integrating, and reasoning over dispersed information without losing long-range dependencies.
- **Interaction Quality:** Ensuring coherent, concise, and natural multi-turn conversations.

To support these heterogeneous objectives, we construct a diverse prompt mixture spanning task categories, languages, interaction formats, and difficulty levels. Furthermore, we employ a **hybrid reward system** to support these heterogeneous objectives. Deterministic constraints use rule-based rewards, while a generative reward model handles nuanced semantic quality dimensions.

4.2.3 Agentic RL

The Agentic RL expert masters long-horizon planning and tool use across various sandbox environments. To ensure real-world adaptability, training environments are treated as black-box systems. During Agentic RL, we adopt several well-used and customized techniques to enhance training stability and sample efficiency, as detailed below:

- **Token-in-Token-out (TITO):** Eliminates tokenization mismatches between trajectory generation and policy updates, stabilizing gradients.
- **Online Dynamic Filtering:** We continuously track per-sample pass rates. Consistently solved samples are dynamically dropped from the training loop, keeping the model perpetually at its learning frontier.
- **Syntax Penalties:** Malformed tool calls (e.g., missing arguments, invalid JSON) trigger immediate, explicit format error penalties in the reward signal.
- **Verifiable Evaluation:** Rewards are deterministic and based on objective execution outcomes outside the training loop, heavily mitigating the risk of reward hacking.
- **Warm-Start Initialization:** Agentic RL initializes from the *General RL* checkpoint, capitalizing on its superior instruction-following and format-correctness baseline.

4.2.4 Code RL

To build robust software engineering and terminal interaction capabilities, we route RL data through a specialized **black-box framework**. A gateway server sits between the agent harness and the inference engine, decoupling agent logic from the RL data flow. It captures token-level data (prompts, completions, logprobs) and reconstructs full trajectories for training—also stabilized by the TITO mechanism.

As coding capabilities scale, so does the risk of reward hacking. We counteract this with an **automated auxiliary monitor agent** that tracks reward distributions and trajectory patterns in real time. It flags anomalies, issues interpretable alerts, and provides actionable diagnostics for rapid reward refinement, improving training stability.

4.3 Multi-Expert On-Policy Distillation

Following the separate optimization of RL expert models, we employ **On-Policy Distillation (OPD)** to fuse their complementary capabilities back into a single, unified model. The distillation prompts are sampled from a mixture of the datasets used during RL expert training. For every prompt, we generate a trajectory using the *current* policy, then query the corresponding domain expert to provide per-token supervision. OPD reuses the RL training objective but replaces the advantage with the per-token log-probability gap between the selected expert and the current policy:

$$A_{i,t} = \text{sg} \left[\log \frac{\pi_{\theta_{\text{expert}}}^{\text{train}}(y_{i,t} \mid x_i, y_{i,<t})}{\pi_{\theta}^{\text{train}}(y_{i,t} \mid x_i, y_{i,<t})} \right],$$

where $\text{sg}[\cdot]$ is the stop-gradient operation, and $\pi_{\theta}^{\text{train}}$ and $\pi_{\theta_{\text{expert}}}^{\text{train}}$ denote the current policy and the selected expert for sample i computed by the training-engine forward pass, respectively. This differs from GLM-5, which reports reading expert probabilities from the inference backend and notes a future migration from $\pi_{\theta_{\text{expert}}}^{\text{infer}}$ to $\pi_{\theta_{\text{expert}}}^{\text{train}}$ [21]. We additionally apply lightweight regularizers on the advantage to focus learning on informative tokens and stabilize optimization: masking tokens with near-zero advantages and clipping outlying log-ratio values.

Compute Optimization. To reduce the NPU resources required for advantage computation, we load experts sequentially into a shared NPU pool, rather than assigning a dedicated NPU pool to each expert. After rollout, samples are grouped by their corresponding expert; for each group in turn, we load the expert’s weights and compute the expert log-probabilities for all its samples with a training-engine forward pass. This avoids keeping all experts concurrently resident while preserving the same per-token scoring procedure.

4.4 Resolving Training-Inference Mismatch

4.4.1 Current Practices

The following paragraphs detail the specific mechanisms we employ to mitigate the training-inference mismatch. As a matter of fact, no single technique suffices across all scenarios; instead, we compose them depending on the model architecture, hardware configuration, and observed mismatch severity.

FP16. In practice, we found that switching from BF16 to FP16 is effective. To address the risk of gradient underflow caused by the limited dynamic range of FP16, we have integrated the mature dynamic loss scaling technique: Before each backpropagation, multiply the loss by the scaling factor S to increase the gradient amplitude; after backpropagation and before gradient clipping, restore the gradient to its true magnitude through unscaling; if any gradient contains Inf or NaN (i.e., an overflow occurs), skip the current optimizer step and halve the S value.

KPop. In our mainline RL training, we largely replace IcePop [52] and conventional Rejection Sampling with KPop [25] as the primary correction mechanism, as it is lightweight and targets a wide spectrum of training-inference mismatch. In practice, we introduce a shadow hyperparameter mechanism that evaluates multiple candidate ϕ values during the initial steps of different training scenarios, allowing us to estimate their masking behavior before committing to a specific setting. When the train-rollout Pearson Correlation Coefficient (PCC) is below 0.99, we choose a stricter KPop mask. Conversely, when the original train-rollout PCC already exceeds 0.99, we use a looser ϕ to avoid unnecessary filtering and preserve the model’s exploration capability. The optimal ϕ is training-dependent and should ultimately be determined together with the reward trajectory, training stability, and other indicators that jointly reflect the benefit of the current KPop setting on both robustness and final performance; the above rules are mainly used as cold-start heuristics for the early RL stage.

Rollout Routing Replay. Our implementation of Rollout Routing Replay (R3) [36] is built on the mechanism proposed by the VeRL[47] + vLLM[32] community, with the following enhancements:

- **Streaming routed experts from the inference engine.** Our inference engine streams routing records incrementally via the existing OpenAI-compatible streaming API. This design amortizes the bandwidth requirement across the generation timeline rather than concentrating it at the end.
- **Scalable data transport.** The community approach embeds routing records directly into DataProto and relies on Ray [37] for data transfer, which becomes a bottleneck at scale. Instead, we only push meta-data into DataProto and use HCCL communication for efficient transfer of routing records, avoiding unnecessary sequence-length padding as well.
- **Masked R3.** In large-scale training, we observe that hidden states exhibit cumulative divergence and severe drift as the MoE layer deepens and the sequence progresses token-by-token. We found that enforcing a full expert replay under such drifted conditions introduces severe distortion to the output logits and causes a drastic decline in the PCC, which could trigger gradient explosion or training divergence. To mitigate this issue, we introduce an adaptive thresholding mechanism to quantify routing drift and dynamically generate a routing replay mask. Replay is deemed undesirable for a token, unless the overlapped expert choices between the ones proposed during training and those recorded from inference exceed a tunable threshold. This method enhances training-rollout consistency, particularly for deep MoE architectures with long-sequence inputs.
- **Cached R3.** Consider agentic RL, which requires multi-round inference. Decoded tokens will go through prefilling in the following rounds. However, subsequent prefilling can yield different expert routing choices for the generated tokens compared to the original generation. Cached R3 addresses this by persisting the expert routing choices recorded at the time each decoded token was *first generated* and replaying those cached choices during training.

4.4.2 Root Cause Analysis of Training-Inference Mismatch

As we experiment with asynchronous RL, we observe that the above mechanisms are not always sufficient to mitigate the mismatch, which motivated a comprehensive root-cause investigation. The analysis below provides the diagnostic foundation for strict-match solutions.

The training-inference mismatch in RL stems from heterogeneous factors that manifest at different levels of the system stack. Through systematic analysis, we classify them into three categories by failure mode: **explicit definition mismatches** (mathematically non-equivalent formulations), **computation graph reordering mismatches** (mathematically equivalent but numerically divergent due to altered operation sequences), and **hardware scheduling non-determinism** (run-to-run variance from micro-architectural execution on the Ascend NPU).

Category A: Explicit Definition Mismatches. High-performance RL training frameworks (such as VeRL) typically adopt a “dual-engine” architecture: the training component is built upon large model training frameworks like Megatron-LM [48], while the inference component integrates highly optimized inference engines like vLLM. This architecture forces the system to maintain two independently defined model definition scripts on each of the training and inference sides. With such two model definitions, subtle discrepancies in low-level model details and configuration parameters are almost inevitable, leading to mathematically non-equivalent computations.

Category B: Computation Graph Reordering Mismatches. Training (throughput/backward-oriented) and inference (latency-oriented) adopt different acceleration strategies that reorder floating-point operations despite identical mathematical expressions, causing numerical divergence. We attribute these to three root causes: (i) *deployment strategy* – selecting different algorithmic variants for the same module, e.g., MLA switching between Absorptive (MQA, decode) and Non-Absorptive (MHA, training/prefill) modes, or MoE experts accumulated in ascending Expert-ID order (training) versus descending Top- k score order (inference); (ii) *parallelism strategy* – training and inference favor different parallelism strategies, which alter both the floating-point accumulation precision and the accumulation order, ultimately causing numerical divergence; and (iii) *fused operator* – kernel fusion implicitly rewrites the underlying computation graph and numerical precision; as training and inference implement the same module with different kernels, this directly introduces numerical mismatches, e.g., distinct FlashAttention [10] families, MoE dispatch/combine kernels, and sampling-op mismatches.

Category C: Hardware Scheduling Non-Determinism. The dynamic execution strategy (Tiling) of kernels constitutes the deepest and most subtle source of inconsistency. To accelerate the Decode phase, inference typically enables features such as dynamic Batching, KV Cache, ChunkedPrefill, or APC. This implies that when computing identical token positions, the training side processes complete 2D/3D tensors, whereas the inference side handles chunked or single-step 1D tensors, failing to maintain consistency across the Batch and Sequence dimensions. The NPU auto-selects tiling strategies by input shape, introducing run-to-run non-deterministic accumulation via: *SplitK* (partitioning K -axis accumulation across AI Cores, e.g., MatMul, FlashDecoding), *K-axis swizzling* (interleaving memory access to improve HBM bandwidth, e.g., MatMul), and *AtomicAdd* (asynchronous multi-core collection with stochastic core arrival, e.g., AllReduce, whose topology-aware algorithms such as Ring/Tree further diverge).

More detailed operator- and input-shape-level implementation differences can be found in Appendix A.6.

4.4.3 Preview of Full-Alignment Solution under the Ascend Architecture

To eliminate these discrepancies on the Ascend NPU and achieve end-to-end training-inference alignment, we designed a layer-by-layer progressive white-box alignment methodology. (*Prerequisite: disable random sampling and enable deterministic communication to freeze generative behaviors, and fully align parallelism on both sides.*)

Our alignment methodology proceeds in two phases of increasing difficulty. **Phase I** targets Prefill, whose full-sequence computation is mathematically close to training forward, resolving Category A and B mismatches. **Phase II** tackles Decode, where single-step, shape-mismatched computation triggers the deeper Category C mismatches and demands kernel-level intervention.

Phase I: Strict Alignment between Training Forward and Inference Prefill (Categories A & B). Prefill shares identical input sequences with training forward, making it the natural starting point for establishing an alignment baseline. As shown in Figure 13 in the appendix, we first developed an Ascend-tailored tensor hook that captures layer-by-layer output tensors during training forward and inference prefill, exporting them as raw binaries. Using high-precision binary discrepancy comparison, we reversely located the numerical bifurcation points and thereby rectified the explicit code and parameter errors of Category A. To converge the operator behaviors of Category B, we further re-implemented FlashAttention for both sides within a single operator (open-sourced in omni-ops²), enabling full consistency between training and inference.

Phase II: Deep Operator Alignment between Inference Decode and Training Forward (Category C). Decode’s “single-step iterations” versus training’s “full-sequence parallelism” cause severe shape discrepancies that trigger Category C non-determinism—unresolvable by configuration alone and requiring direct modification of operator implementations. To bridge this gap, as illustrated in Figure 14 in the appendix, we first constructed a “original prompt + first token” input on the training side, breaking long sequences into single-step structures that align the semantic boundaries between training and inference. We then exploited open-source CANN operators to directly modify the kernel Tiling strategies, constraining optimization behaviors such as SplitK and unifying the accumulation axis across multi-core parallelism, after which we recompiled and deployed custom installation packages, eliminating the underlying numerical drift caused by abrupt shape variations.

Together, these two phases cover the full spectrum of training-inference mismatch categories along a deliberate, depth-ordered trajectory—from surface-level code discrepancies, through operator behavioral divergence, down to kernel-level numerical drift—with each layer of the stack pinned against its matching counterpart. This progression ultimately yields sufficient alignment between training and inference under a fixed parallelism configuration, providing a fully white-box, reproducible alignment baseline for subsequent deployment on the Ascend NPU.

²<https://gitee.com/omniai/omni-ops>

5 Evaluation

We evaluate openPangu-2.0 across a variety of public benchmarks, assessing its general, reasoning, agentic, and coding capabilities. openPangu-2.0 demonstrates competitive performance across a broad spectrum of general capabilities as well as STEM reasoning tasks. On agentic benchmarks, openPangu-2.0-Pro delivers proficient in skill-use, agentic search, and a variety of Claw-oriented tasks. For coding tasks, while the model performs well on basic code QA and feature implementation, there remains a noticeable gap compared to top-tier models when handling complex, real-world software engineering workflows. Beyond public benchmarks, we leverage our in-house evaluation harness to demonstrate openPangu-2.0’s practical utility. Specifically, we showcase its capacity to execute end-to-end deep research with interactive visualizations and to autonomously generate full-stack applications.

5.1 Evaluation On Public Benchmarks

5.1.1 General and Reasoning

For general capabilities, **Instruction Following** is evaluated using IFEval [64], IFBench [40], AgentIF [41], and SysBench [42], covering verifiable constraints and complex system-level instructions. **Factual Reliability** is assessed using SimpleQA Verified [26] and Chinese-SimpleQA [27], which measure short-form factual accuracy and hallucination. **Long-Context Understanding** is evaluated using CL-Bench [20], with an emphasis on learning and reasoning from task-specific contexts. Finally, **Multi-turn Interaction** is assessed using MultiChallenge [17], which tests instruction retention and consistency across multiple conversational turns. We evaluate reasoning performance on the text-based subset of Humanity’s Last Exam (HLE) [3], AIME 2026 [63], HMMT 2026 [4], IMO-AnswerBench [35], Apex [16], GPQA-Diamond [44], and BBEH [31].

From Table 5, we can see that both openPangu-2.0-Flash and openPangu-2.0-Pro achieve strong performance across a wide range of general benchmarks, covering in-context learning, instruction following, and multi-turn comprehension. For world knowledge and factual reliability (SimpleQA Verified, Chinese-SimpleQA), openPangu-2.0-Pro demonstrates strong results, outperforming openPangu-2.0-Flash. On STEM reasoning tasks, openPangu-2.0-Pro delivers competitive performance, and it substantially outperforms openPangu-2.0-Flash on complex tasks such as Apex, HLE, IMO-AnswerBench, and HMMT 2026.

5.1.2 General Agent and Coding

For agentic benchmarks, **Agentic-Tool/Skill-Use** is evaluated on Tau2-Bench [6], MCP-Atlas [5], SkillsBench [33]. **Claw-oriented and Real-world Tasks** are evaluated w.r.t. PinchBench [2], Wild-ClawBench [19], Claw-Eval [61], GDPVal [39], **Agentic-Search** is evaluated on BrowseComp [54]. SkillsBench is evaluated using OpenCode as the harness. For Claw-Eval, only the non-multimodal splits are evaluated. For GDPVal, we conducted an internal evaluation using the public GDPVal dataset and rubrics with point-wise task-level scoring on GDPVal-AA v2 scaffolding. For BrowseComp, we employ the discardall context management strategy as in [15] with a maximum of 300 iterations. For coding task, LiveCodeBench V6 [29], FeatBench [8], DeepCodeBench [9], and SWE-bench Verified [30] are evaluated regarding code function generation, feature implementation, QA, and software engineering respectively. DeepCodeBench and FeatBench are evaluated using OpenCode scaffolding, and SWE-bench Verified is evaluated using OpenHands scaffolding.

From Table 5, we observe that openPangu-2.0 demonstrates proficient capabilities in skill-use, agentic search, and various Claw-oriented tasks. The model also performs well on basic code QA (DeepCodeBench) and feature implementation (FeatBench) tasks. However, we also notice that our complex agentic and coding capabilities still exhibit certain gaps compared to other first-tier models.

In our in-house experiments, we observe that establishing a strong baseline in general knowledge and reasoning is helpful to learn more sophisticated agentic and coding behaviors. Reciprocally, training within multifaceted agentic environments strengthens basic competencies, including instruction following, long-trajectory context processing, multi-step reasoning, and procedural planning. Finally, we found that scaling high-quality, diverse long-trajectory agentic data and interactive environments across both mid-training and post-training is highly advantageous, establishing one of the core strategies to scale in future model releases.

Table 5: Evaluation results of openPangu-2.0-Flash and openPangu-2.0-Pro across general, reasoning, agent, and coding benchmarks. Both the Non-Thinking and Thinking inference modes are reported. Results in bold indicate the best performance across all openPangu-2.0 variants. All Evaluations are conducted with decoding setting topp 0.8 and temperature 1.0.

Benchmark	openPangu-2.0-Flash		openPangu-2.0-Pro	
	Non-Thinking	Thinking	Non-Thinking	Thinking
<i>General</i>				
CL-Bench _(Acc)	15.5	20.4	17.8	19.7
IFEval _(PromptStrict)	89.3	95.9	90.4	94.5
IFBench _(PromptStrict)	54.4	79.6	59.5	79.9
AgentIF _{((CSR+ISR)/2)}	43.9	44.9	42.2	44.4
SysBench _(ISR)	87.9	91.1	82.6	90.8
Multichallenge _(Acc)	51.9	68.4	52.2	62.9
SimpleQA Verified _(Acc)	21.6	24.5	27.1	34.9
Chinese-SimpleQA _(Acc)	66.0	66.7	75.7	74.2
<i>Reasoning</i>				
HMMT Feb 2026 _(Avg@16)	63.2	76.5	63.3	86.2
– w/ Python	–	90.7	–	93.2
AIME 2026 _(Avg@16)	86.5	93.3	87.3	95.4
– w/ Python	–	98.1	–	97.9
Apex _(Avg@16)	–	1.0	2.1	17.7
IMO-AnswerBench _(Acc)	62.3	76.5	60.8	84.3
BBEH _(HarmonicMean)	51.5	62.5	55.0	69.7
GPQA-Diamond _(Avg@4)	79.8	83.7	81.1	87.9
HLE _(Acc)	8.5	20.3	9.0	27.1
<i>Agent</i>				
TAU2-Bench _(Avg@3)	74.0	88.0	71.6	81.1
MCP-Atlas _(Acc)	47.9	58.9	48.8	61.3
SkillsBench _(Avg@5)	40.0	42.6	47.5	48.9
PinchBench _(Avg@3)	82.5	85.6	88.9	84.0
WildClawBench _(Avg@3)	35.0	41.5	43.1	46.5
Claw-Eval _(Pass@3)	58.2	57.7	49.4	61.7
GDPVal _(Acc)	–	51.4	51.2	74.9
BrowseComp _(Acc)	–	57.0	–	65.7
<i>Coding</i>				
LiveCodeBench V6 _(Avg@3)	50.9	85.1	74.5	85.7
FeatBench _(Avg@3)	45.8	45.9	48.4	47.1
DeepCodeBench _(Avg@3)	70.9	76.5	72.6	74.2
SWE-bench Verified _(Avg@3)	57.6	63.1	66.8	68.5

5.2 Evaluation On In-house Harness

5.2.1 Deep Research and Interactive Visualization

Deep Research has emerged as one of the most prominent applications of modern LLM agents. While existing public benchmarks primarily emphasize the depth or breadth of information retrieval, practical utility hinges not only on what an agent discovers, but on how effectively it transforms raw findings into accessible, actionable knowledge. Crucially, as LLMs advance in both agentic reasoning and code generation capabilities, a fundamental modality shift is underway: knowledge synthesis is transitioning from static text to dynamic, visual, and interactive artifacts. By leveraging execution-backed coding to analyze, render, and present information on the fly, LLM agents empower humans to acquire, interrogate, and communicate complex insights with unprecedented efficiency.

To study this end-to-end capability, we deploy openPangu-2.0-Flash within an in-house research-and-delivery harness. The harness enables the model to use search tools, coding tools, and a visual frontend-development environment. Rather than returning only a sequence of textual paragraphs, it produces a functional website that presents the research findings through coordinated text, visuals, and interactions.

Showcases can be found on <https://op-agent.atomgit.com/>, spanning diverse real-world application scenarios including comprehensive industry surveys, academic literature reviews, and tutorial generation. Demonstrating this vision in practice, openPangu-2.0 goes beyond simple textual summaries to generate high-fidelity statistical charts, intricate scalable vector graphics (SVGs), and crystal-clear conceptual schematics. Crucially, the model excels at creating dynamic content, including smooth informative animations and fully responsive interactive components, thereby transforming static information into an immersive knowledge exploration experience.

5.2.2 Full-Stack Mini-App Generation

Unlike frontend-only webpage generation, full-stack app generation is a more demanding agentic task: the agent must wire together the frontend, backend, database, real-time services, and deployment, while balancing visual and interaction quality against end-to-end functional correctness. Beyond traditional desktop applications, generative mini-apps on mobile platforms—particularly those built upon native toolchains like HarmonyOS—herald a fundamental paradigm shift in human-computer interaction.

To rigorously evaluate openPangu-2.0 on this challenging task, we establish a programmatic-first assessment pipeline focused on end-to-end functional correctness and interaction quality. Benchmark queries are decomposed into atomic functional claims, each annotated with a verification type and criticality level. For every generated application, a computer-use agent actively interacts with the application interface to gather multimodal evidence, including UI trees, rendered screenshots, and runtime traces. A multi-stage verification cascade checks these signals, invoking a vision-language judge for perceptual or subjective attributes, and aggregates the outcomes into a criticality-weighted functionality score.

Through this comprehensive evaluation framework, openPangu-2.0 demonstrates exceptional end-to-end generation capabilities across diverse real-world domains, reliably maintaining high functional correctness alongside responsive interaction quality. Concrete showcases can be explored on <https://op-agent.atomgit.com/>. Representative examples include personalized travel itinerary planners with integrated route mapping and interactive schedules, as well as targeted stock tracking assistants featuring real-time data pipelines and dynamic charts. These applications demonstrate that openPangu-2.0 can reliably transform high-level user intent into production-ready, interactive native mini-apps—laying a solid foundation for the future of on-demand, mobile-first software synthesis.

6 Inference

The openPangu-2.0 series adopts a large-sparse Mixture-of-Experts (MoE) architecture paired with a hybrid attention mechanism (SWA:DSA ratio of 2:1), yielding substantial advantages for long-sequence tasks. To maximize serving efficiency on Ascend hardware, we introduce a comprehensive optimization suite. These techniques fall into five core categories:

- **Operator Innovations:** Implementation of novel fused operators, including a specialized mHC kernel for inference and Pangu-specific Attention Sink operators.
- **Parallelism & Execution Paradigms:** Introduction of new parallelism models, such as Felix CP and per-core multi-stream execution.
- **Cache Management:** A hybrid KV-cache system featuring Omni Cache, pioneering APC, and Multi-MTP support tailored for ModAttn (a Mamba-like architecture).
- **Framework Enhancements:** System-level upgrades including full-graph Multi-MTP and Lossless Parallel Tokenization (LoPT).
- **Quantization:** A holistic quantization scheme designed to balance accuracy, operator efficiency, and parallel processing.

Together, these integrated designs unlock exceptional performance on the Ascend 910C:

- **openPangu-2.0-Flash:** Achieves a 128K ultra-low latency of 5.63 ms TPOT and a throughput of 1,846 tokens/s per NPU (at 15 ms TPOT).
- **openPangu-2.0-Pro:** Achieves a 128K ultra-low latency of 9.55 ms TPOT and 1,326 tokens/s per NPU (at 20 ms TPOT).

6.1 Deployment

We introduce two inference configurations tailored to distinct operational demands: an ultra-low latency variant prioritizing extreme single-request responsiveness, and a low-latency tier balancing throughput and decoding latency for general-purpose workloads.

The ultra-low latency configuration adopts a co-located (PD-mix) deployment, applying layer-specific parallelization: TP for SWA layers, EP for MoE layers, and a dynamic strategy for DSA layers that utilizes CP during prefill and switches to TP via redundant weights during decode. Conversely, the low-latency version employs a PD-disaggregated architecture. Its prefill node mirrors the exact parallelization topology of the ultra-low latency variant, while the decode node couples Attention DP with MoE Expert Parallelism to optimize overall throughput at scale.

6.1.1 Quantization Format

We apply W8A8 integer quantization to openPangu-2.0 via statically symmetric output-channel weight quantization and dynamic per-token activation quantization. Taking the Pro variant as an example, this compresses the weight footprint from approximately 1009 GB to 517 GB, achieving a $1.95\times$ reduction. Calibration leverages channel-wise smoothing—dividing activations and multiplying weights by s to preserve layer outputs—while relaxing SmoothQuant’s[55] rigid $a + b = 1$ constraint via a 2D search over decoupled exponents (a, b) . For the critical up projection, input- and output-side smoothing coefficients are iteratively optimized in alternating rounds until convergence.

Whether to quantize individual modules is determined by jointly evaluating memory savings, operator capability, precision sensitivity, performance, and multi-stream scheduling. While large layers yield optimal compression gains, several factors restrict specific components to BF16: fused operators restricted to BF16 inputs, layers preceding discrete selections (e.g., TopK routing and indexing) sensitive to quantization noise, error-accumulating residual paths, small layers where overhead outweighs savings, and operators already hidden by side-stream windows. The complete per-module decision mapping is detailed in A.8 (Table 11).

6.2 Ascend Operator Performance Optimization

The total execution time of an operator typically comprises device time (compute or memory-access), kernel launch overhead, and host scheduling overhead. In low-latency and ultra-low-latency scenarios, kernel launch overhead and host scheduling now account for an increasingly larger fraction of the overall execution time, so we design multiple fused and customized operators that work in concert with graph-mode multi-stream parallelism.

6.2.1 Compute Operator Optimization

Inference mHC Operator Optimization. The mHC computation replaces the traditional residual stream and runs through every layer’s Attention and MoE modules [57]. The DeepSeek design computes H_{pre} , H_{post} , and H_{res} via three fused operators, but for low-latency inference it has two issues: (i) the 24-column FP32 matrix product $\bar{x}W_{\text{mHC}}$ ($n = 4$ scheme, $24 = n^2 + 2n$) has only 4 columns consumed immediately while the remaining 20 are deferred to the final 4-way mHC merge, carrying redundancy on the critical path; (ii) the computation of H_{pre} together with surrounding RMSNorm and the previous layer’s mHC merge involves too many operators, causing kernel launch overhead and repeated movement of intermediate variables.

We redesign the mHC computation into three parts: (1) *mhc_sandwich_norm_post_preonly*, which factors out only the H_{pre} -related computation and fuses it with surrounding operators into one large fused operator; (2) *mhc_pre_split_post_res*, which implements the remaining mHC computation as a single fused operator with low scheduling priority, overlappable via multi-stream parallelism; (3) a

dedicated *sinkhorn* operator, likewise overlappable. Together, the first operator covers the H_{pre} path (including surrounding RMSNorm and the previous layer’s mHC merge), while the second covers H_{post} and H_{res} , significantly improving mHC performance in low-latency inference.

SWA Operator Optimization. In the SWA architecture, a Parametric Attention Sinks mechanism mitigates representation degeneration from attention scores focused on the first token. We propose an intra-core dynamic micro-tile-level mask-skip algorithm: the coarse-grained basic block on L1 is split into micro-tiles aligned with the compute array’s physical dimensions, and “all-invalid” micro-tiles are bypassed before L1-to-L0 movement, eliminating redundant compute and memory access. This scheme keeps Parametric Attention Sinks resident in L1, offloads tiling scheduling to AI CPU, and dynamically adjusts basic-block granularity for multi-core load balancing. The SWA operator achieves a 30% ~ 40% performance improvement.

In addition to the operators discussed above, we also perform detailed performance optimization on the DSA, LI, and MoeInitRouting operators tailored to Pangu-specific characteristics. The source code of these operators will be open-sourced alongside this work.

6.2.2 Communication Operator Optimization

Communication efficiency directly affects end-to-end performance. We optimize communication at two levels: first, selecting appropriate parallelization strategies and collective algorithms; second, reducing operator-launch, synchronization, quantization, and data-movement overhead after the communication pattern is determined. This section presents three key techniques for single-node, low-latency scenarios—TP AllReduce over EP AlltoAllv, Non-BSP asynchronous pipeline via atomic ordering, and reducing computation and memory overhead within communication.

TP AllReduce over EP AlltoAllv for Ultra-Low Latency. In the ultra-low latency scenarios, we replace the two AlltoAllv operations of conventional EP (dispatch + combine) with redundant local gating and a one-shot AllReduce after attention TP, which is beneficial provided that the added communication and computation cost is smaller than one RTT. On the ultra-low latency openPangu-2.0 Flash deployment, this optimizes end-to-end latency by about 20%.

Non-BSP Dispatch via Atomic Ordering. In Dispatch, each token’s write position depends on preceding tokens’ target-expert info, so a slow rank drags down the entire reorder. We remove full-core synchronization via a Non-BSP asynchronous pipeline: the write offset is obtained by a hardware atomic increment (AtomicAdd, one counter per expert) with no inter-core negotiation, and fusing AtomicAdd with Store uses push semantics to compute the destination address on the receiver side. Atomic ordering optimizes Dispatch latency by 15%–20% (5 μ s on a 16P/HiddenSize-4096 setup).

Reducing Computation and Memory Overhead in Communication Operators. Conventional MoE Dispatch repeatedly reads and quantizes each token for every routed expert. We instead adopt a token-centric execution model, where each token is read and quantized once before being dispatched to all routed experts. This removes Top- K -dependent redundant computation, reducing Dispatch latency by 25%–30% and improving end-to-end latency by up to 5%.

6.2.3 Graph Optimization

In low-latency inference, CANN provides the ACL Graph acceleration scheme (the Ascend counterpart of CUDA Graph) and Npugraph_ex, a CANN-based PyTorch FX graph-optimization component that connects to ACL Graph through the `torch.compile` entry. Npugraph_ex provides two complementary features: **static kernel compilation**, which ahead of time completes scalar computation, memory-address derivation, and loop-bound determination so the runtime avoids repeated parameter parsing and synchronization-point insertion; and **Superkernel**, which fuses multiple consecutive or logically related kernels into one unified super-kernel via sub-function calls to reduce launch count. In the openPangu-2.0 ultra-low-latency scenario, enabling both yields a TPOT improvement of 3.2 ms (1.2 ms from static kernel compilation, 2 ms from Superkernel).

6.3 Parallelism Strategy

As discussed in Section 6.1, the MoE part of openPangu-2.0 adopts the classic EP deployment on Atlas A3. The Attention part uses DSA CP + SWA TP on the prefill side and Attention DP on

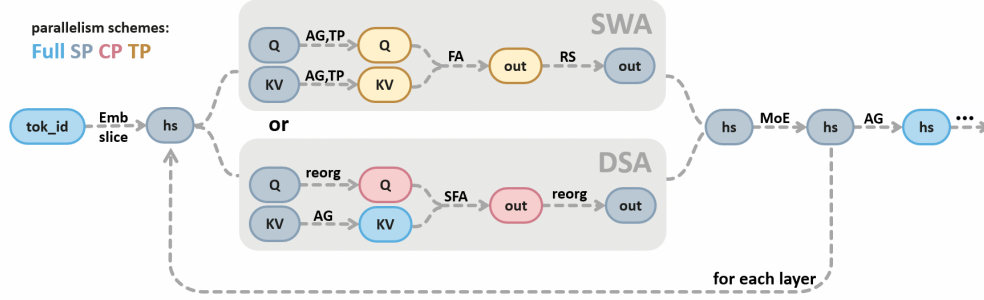


Figure 5: Full forward flow of the Felix CP scheme.

the decode side, with the prefill-side Attention deployment being the most complex—this also buys significant performance benefits, bringing out the model’s high-performance characteristics on long sequences. Throughout the model’s computation we adopt multi-stream parallel strategies, including communication and computing, Cube and Vector, and cross-AI-Core multi-stream, to maximize exploitation of the Ascend hardware.

6.3.1 Felix CP: Attention Module Parallelism Strategy

The attention part of openPangu-2.0 adopts the hybrid form of DSA + SWA + ModAttn. DSA and SWA differ fundamentally: DSA is memory-bound with irregular access and Indexer is compute-bound with $O(S^2)$ complexity, where CP-zigzag linearly amortizes the $O(S^2)$ workload with no extra communication; SWA is compute-bound with regular contiguous access within a fixed window $O(S \cdot W)$, where TP efficiently utilizes the Cube unit. Since the Attention module’s input and output are both in SP format, we apply SP to parts of the Attention Prolog / Epilog (including ModAttn), eliminating redundant computation on each card and reducing the communication from format conversion (e.g., CP→SP or TP→SP near Out-Proj). Based on this analysis, we propose the Felix CP scheme, shown in Figure 5 for the hybrid attention architecture of openPangu-2.0.

DSA CP. The DSA layer adopts the absorbed-weight mode: receives SP-state input and converts to CP state in the Q and w branches inside the Indexer, so Q_{down} , KV_{down} , and the Indexer compute only the rank’s assigned sequence, and SFA transfers only the kv_cache corresponding to Q—all reducing redundancy vs. TP.

SWA TP. The SWA layer combines TP with the non-absorbed-weight mode: AllGather is placed after Q/KV-ModAttn (lower data dimensions) to reduce communication, while $Q_{down}/KV_{down}/Q$ -ModAttn/ KV -ModAttn use SP; Q_{up}/KV_{up} adopt TP (Full→TP) for Attention; Attention output is converted TP→SP via All2All, and O-ModAttn with fully loaded O_{proj} completes the module in SP.

ModAttn-SP. ModAttn SP relies on two primary components: local metadata construction and parallel computation. During the metadata phase, each rank independently processes prefix-cache block information, and maps local KV blocks to the global state. The computation phase then retrieves the rank’s running cache from the global KV pool, and executes the fused convolution kernel before globally synchronizing the updated caches. Crucially, this represents the first open-source SP implementation supporting APC for Mamba-like architectures, and generalizes to other Mamba-like networks.

6.3.2 Multi Stream

openPangu-2.0 implements a dual-stream architecture with explicit core-budget control to maximize NPU utilization. Each Ascend 910C die integrates 24 Cube cores (AIC) and 48 Vector cores (AIV). Under single-stream scheduling, operators run sequentially, leaving specialized compute cores idle despite non-conflicting resource needs. We reclaim this capacity using three complementary mechanisms:

- **Core-budget control.** Allocating all cores to a single operator is inefficient on the decode path: small-shape Vector kernels incur high per-core launch overhead, and concurrent streams contend for physical cores. We cap each operator’s AIC/AIV occupancy so both streams fit within die bounds.
- **Cube/vector parallelism.** The MLA prologue features naturally complementary workloads: the Q path is dominated by Cube-heavy GEMMs, whereas the KV/mHC path consists primarily of Vector operations. Running these on separate streams keeps AIC and AIV pipelines active simultaneously. Furthermore, we split the large Q projection into two balanced sub-projections across streams, shortening the critical path.
- **Communication/compute overlap.** In MoE blocks, we offload shared-expert computation to the side stream, overlapping its execution with the routed pipeline and its communication tail [62].

6.4 KV Cache Management

6.4.1 Omni Cache

Contemporary inference engines maintain the KV block pool in HBM, which incurs two fundamental limitations. **HBM resource contention:** the block pool competes with model weights for scarce HBM, so once weights are loaded, a single A3 device lacks the residual capacity to hold the KV of openPangu’s 512K-token sequences. **Inadequate KV capacity for long contexts:** the HBM left for KV accommodates only a handful of long-context requests, so historical KV is evicted almost immediately, driving APC hit rates toward zero and limiting decode concurrency.

A key observation motivates our design. We distinguish **KV buffers**—the subset of KV actively read by attention in the current forward pass—from the **KV cache**—the remainder, comprising historical KV and unoccupied slots. Only KV buffers demand HBM bandwidth; the KV cache merely consumes capacity. This distinction admits a **DRAM-centric** architecture in which the KV cache resides in host memory at terabyte scale, while KV buffers occupy HBM only during prefill—reduced to a single layer-shared buffer reused across layers and held for each layer’s computation.

DRAM centric KV management. Figure 6 contrasts the memory layout with existing HBM-based block pools.

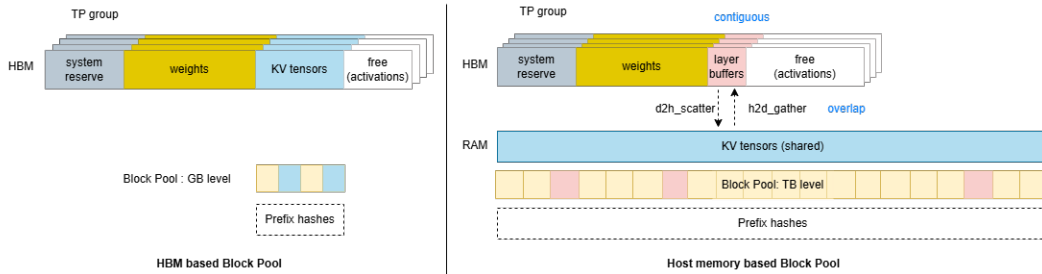


Figure 6: Memory layout comparison. Left: contemporary HBM-based architecture—Page pool in HBM with full layer allocation. Right: Omni Cache—KV storage moves to a shared host memory pool, HBM KV buffers for active computation, and layer-wise D2H scatter / H2D gather overlap with compute to hide transfer latency.

With openPangu, Omni Cache enlarges native KV space by more than 10× and APC hit rate by 3×. The released HBM allows the prefill node to deploy on a single A3, with max-batch-tokens increased 4× while still supporting 512K long sequences.

6.4.2 Hybrid Attention and KV Cache Management

The openPangu-2.0 series exposes a heterogeneous attention structure: DSA keeps compressed attention cache plus indexer state; MLA/SWA stores shared compressed KV and RoPE; ModAttn is a Mamba-like boundary state for three causal convolution modules (Q / compressed-KV / output

paths). The design goal is to retain vLLM’s Single BlockPool abstraction: all cache-bearing layers draw from a single block-oriented pool and participate in one common prefix-cache negotiation, while each backend interprets the block payload according to its own kernel layout.

Page-Size Alignment. The central mechanism is to align the physical page size of the three cache families, distinguishing the real payload from the padded page. For MLA/SWA the payload is $B_{\text{MLA}} = \text{block_size} \cdot (512 + 64)$ (shared compressed KV + RoPE); for DSA it additionally contains the indexer state, $B_{\text{DSA}} = \text{block_size} \cdot (512 + 64 + 128)$; for ModAttn the payload is $B_{\text{ModAttn}} = (K - 1 + N_{\text{spec}}) \cdot (q_{\text{lor}} + kv_{\text{lor}} + H \cdot d_v)$, where K is the convolution width, N_{spec} the speculative-token reservation, and the three terms are the Q, compressed-KV, and output-side states. The implementation is quantization-aware: DSAAttentionSpec computes the real page size from `cache_dtype_str`, so FP8/INT8/HiF8/LI-only layouts attach scale tensors without changing the scheduler interface. The runtime chooses a common padded page size large enough for every active type. Padding is a deliberate trade-off: it may waste a small amount of memory inside a page, but prevents the scheduler, block pool, and prefix-cache coordinator from carrying model-specific allocation cases.

Strided Cache Views and KV Specs. The aligned page is allocated as one raw memory tensor, and each backend reinterprets it via `torch.as_strided` views whose first stride jumps by the full padded page size, automatically skipping padding. The vLLM interface is extended through three Pangu-specific KV cache specs: DSAAttentionSpec (understands DSA payload and quantized formats), ShareKVSlidingWindowSpec (models MLA/SWA as sliding-window with shared KV storage), and MomeSpec (extends vLLM’s MambaSpec, recording three state shapes/dtypes, convolution kernel size, and speculative-token reservation). The corresponding managers keep the rest of vLLM unchanged: DSA uses the normal full-attention manager, MLA/SWA uses a sliding-window manager for shared KV, and ModAttn uses MomeManager, which treats the cache as a boundary-state cache (like Mamba, only the latest valid convolution boundary state is needed, not all historical states).

Prefix Caching Across Heterogeneous Groups. Hybrid prefix caching requires every cache group to agree on a single reusable prefix length via vLLM’s fixed-point negotiation (candidate hit length decreases until all groups agree). For full-attention groups such as DSA, cache lookup is downward-closed, so the manager scans left to right and returns a contiguous sequence of cached blocks. For ModAttn with APC enabled, the cache blocks extend to $\lceil L/\text{block_size} \rceil$; we group tokens into `block_size`-token blocks and cache the last $(K - 1)$ tokens of each block as prefix caches. Since a ModAttn block stores the boundary state needed to resume computation, MomeManager searches from right to left and stops at the latest available block; earlier positions are filled with null blocks so the logical hit length remains `num_blocks * block_size`, while the physical live state may be only the final ModAttn boundary block.

For MTP or EAGLE-style speculative decoding, full-attention groups may drop the final hit block so hidden states can be recomputed for speculative verification; applying the same rule to ModAttn would be incorrect, because ModAttn reuse depends on the latest valid state rather than a contiguous sequence of hidden states. The Pangu coordinator therefore passes the EAGLE flag only to full-attention groups; ModAttn groups continue to participate in the same fixed-point negotiation, but their hit logic remains Mamba-like.

ModAttn with Support for MTP and APC. The complexity of managing the ModAttn cache peaks when MTP and APC operate concurrently, as unverified speculative tokens can temporarily populate prefix cache blocks before being verified or overwritten. To address this, our framework tracks the number of accepted speculative tokens across scheduling steps, and applies a dynamic correction to guarantee accurate cache-read positioning. This represents the first open-source framework for Mamba-style architectures capable of simultaneously supporting both MTP and fine-grained APC, offering a generalized solution that readily extends to architectures such as Gated DeltaNet [60] and Deepseek Compressed Sparse Attention [15].

6.5 Inference Framework Optimization

In addition to hybrid KV cache management, we introduce three complementary optimizations in the openPangu NPU inference engine to minimize end-to-end latency: (a) fine-grained asynchronous scheduling to eliminate device-side bubbles, (b) a Multi-MTP Full-Graph mode that guarantees

every decode step executes within a static graph, and (c) LoPT, a lossless parallel tokenizer that significantly reduces the TTFT.

Asynchronous Scheduling Optimization for Ascend NPU Inference. We adapt vLLM V1’s asynchronous scheduling framework to our Hybrid Attention model (with MTP-3 enabled) for large-scale EP and DP deployment. In this setup, we identify three critical synchronization bottlenecks: (i) the variable acceptance-length dependency introduced by MTP verification, (ii) frequent CPU-side Tiling invocations for multiple KVCache groups, and (iii) blocking DP-level batch-coordination communication. To address the first bottleneck, we eliminate the speculative-decoding D2H blocking point by returning sampling results and draft tokens via an independent asynchronous copy stream. By employing event pipelining—i.e., chaining the completion notification of the current copy with the launch of the succeeding computation kernel—we fully overlap the CPU waiting time with NPU computation. For the second bottleneck, we offload CPU-side Tiling to the on-chip AICPU (ARM core). This core directly accesses the sequence-state tensors residing in NPU HBM and asynchronously writes the Tiling metadata into the buffer. This design eliminates all D2H and H2D round-trips and operates concurrently with AICore computations. Finally, for DP synchronization, we substitute the blocking CPU-side Gloo AllReduce with a dedicated HCCL ProcessGroup operating in AICPU mode (i.e., offloading collective communication to the on-chip ARM core). In this setup, the entire H2D–AllReduce–D2H chain executes fully asynchronously on the ARM core. Consequently, it consumes zero AICore cycles and never stalls the EngineCore thread.

Multi MTP and Full Graph. Supporting speculative decoding within NPU Graph Mode introduces three compounded engineering challenges, which we address as follows: (1) Multi-MTP integration: we complete the missing framework-side calling logic to ensure proper execution flow into the designated MTP layer. (2) Full-Graph support: beyond the conventional Decode-Only mode, our omnipnu runtime introduces a Full-Graph mode. In this mode, even requests with very few tokens (e.g., 1–3 tokens) are forced into the static graph, avoiding costly graph-capture recompilation and guaranteeing that decode steps consistently run in graph mode under PD (Prefill-Decode) disaggregated deployment. To realize this, operators natively accept TND ($\text{Token} \times \text{Num_heads} \times \text{Dim}$) inputs with dynamic boundaries via `query_start_loc`, while the drafter runs in padded mode with adjusted graph-control logic. (3) Communication optimization: in multi-MTP padded mode, the draft model’s input shape exactly aligns with the target model’s, which allows us to safely skip the pre-draft DP all-reduce. As a result, each decode step saves one all-reduce operation, yielding a per-step performance gain that scales linearly with the DP size.

LoPT: Lossless Parallel Tokenization Acceleration Framework. As context length grows (up to 1M tokens), the inherently sequential nature of mainstream tokenizers (BPE, WordPiece) makes tokenization a non-negligible bottleneck. Naive chunked parallel tokenization suffers from the Boundary Artifact: the same substring may tokenize differently at different positions. We propose LoPT [45], which retains overlapping splits but replaces “Token ID matching” with “character-position matching”—recording each token’s character start/end positions and matching by position rather than ID, fundamentally resolving context-induced ambiguity. LoPT produces output strictly identical to serial tokenization (100% accuracy across code, natural language, and multilingual texts) and achieves 3–5 $\times$ speedup on a 16-core CPU, with overlap overhead below 5%. LoPT is deeply integrated with Token-level APC, avoiding single-token misalignment that would cause APC hash matching to fail. Details in Appendix A.9.

6.6 Performance Results

6.6.1 Theoretical Analysis of Model Architecture

We analyze the attention efficiency of openPangu-2.0’s hybrid DSA:SWA architecture under long-context prefill and decode workloads.

Attention Efficiency. Long-context serving overheads are dominated by prefill compute, resident KV-cache memory, and decode cache traffic. We deploy a 1 : 2 ratio of Dynamic Sparse Attention (DSA) to Sliding Window Attention (SWA): SWA restricts attention to a fixed window W , while DSA scans global context via a lightweight Lightning Indexer and executes core attention only on top- K selected tokens.

Prefill. Long-context prefill latency scales as $t_{\text{attn}}^{\text{P}}(S) \approx \alpha_{\text{P},2} S^2 + \alpha_{\text{P},1} S + b_{\text{P}}$. In our hybrid stack, SWA and core DSA attention scale linearly as $\mathcal{O}(SW)$ and $\mathcal{O}(SK)$; only the Lightning Indexer retains quadratic scaling. Dimension-only accounting yields:

$$\frac{\alpha_{\text{P},2}^{\text{Hybrid}}}{\alpha_{\text{P},2}^{\text{Full}}} \approx \frac{1}{3} \cdot \frac{N_{\text{head}}^{\text{Indexer}}}{N_{\text{head}}} \cdot \frac{D_{\text{Indexer}}}{D_{\text{QK}}^{\text{MHA}} + D_{\text{V}}^{\text{MHA}}} = \frac{1}{3} \cdot \frac{1}{2} \cdot \frac{128}{320} = \frac{1}{15}$$

This 1/15 ratio isolates the dominant quadratic attention-FLOP coefficient rather than end-to-end prefill latency.

KV-cache memory. Resident KV-cache memory grows as $M_{\text{cache}}(S) \approx \alpha_{\text{M}} S + b_{\text{M}}$. SWA layers retain only W tokens, whereas DSA layers store full MLA KV caches alongside compact index keys. At equal precision, memory growth drops significantly:

$$\frac{\alpha_{\text{M}}^{\text{Hybrid}}}{\alpha_{\text{M}}^{\text{Full}}} \approx \frac{1}{3} \left(1 + \frac{D_{\text{Indexer}}}{D_{\text{KV}}} \right) = \frac{1}{3} \left(1 + \frac{128}{576} \right) \approx 40.7\%$$

An idealized 8-bit hybrid cache versus a 16-bit baseline halves this to $\approx 20.4\%$. Actual savings also depend on quantization metadata and runtime layout.

Decode. Long-context decode is cache-traffic bound: $t_{\text{attn}}^{\text{D}}(S) \approx T_{\text{cache}}^{\text{D}}(S)/\text{BW}_{\text{HBM}} + t_{\text{select}}^{\text{D}}(S) + b_{\text{D}}$. While full attention scans every layer’s complete KV cache, SWA reads only W recent tokens and DSA scans compact index keys before gathering K KV entries. Because W and K are independent of S , the leading cache-scan coefficient becomes:

$$\frac{\alpha_{\text{T}}^{\text{Hybrid}}}{\alpha_{\text{T}}^{\text{Full}}} \approx \frac{1}{3} \cdot \frac{D_{\text{Indexer}}}{D_{\text{KV}}} = \frac{1}{3} \cdot \frac{128}{576} \approx 7.4\%$$

This yields a $13.5\times$ reduction ($\approx 3.7\%$, or $27\times$, using an 8-bit hybrid cache vs. a 16-bit baseline). To be specific, at a sequence length of 128K, openPangu-2.0-Pro only incurs a per-batch KV cache read traffic of 0.33 GB, with the total cache size reaching 2.68 GB.

These estimates isolate length-dependent attention terms; end-to-end speedups remain subject to selection overheads, sparse gathering, projections, batching, communication, and hardware utilization.

6.6.2 Latency and Throughput

Ultra-Low Latency. The ultra-low-latency configuration adopts an A3 single-node co-located deployment. The sequence lengths are 128K input + 1K output / 8K input + 1K output, with a global concurrency of 1. Under real data, the Flash model’s TPOT is within 6 ms and the Pro model’s TPOT is within 10 ms. Performance details are shown in Table 6, where ISL = Input Sequence Length and OSL = Output Sequence Length:

Table 6: Ultra-low-latency for openPangu-2.0.

Model	ISL + OSL	TPOT (ms)
openPangu-2.0-Flash	8K+1K	5.27
openPangu-2.0-Flash	128K+1K	5.63
openPangu-2.0-Pro	8K+1K	9.53
openPangu-2.0-Pro	128K+1K	9.55

Low-Latency Throughput. For higher throughput, the low-latency configuration adopts a PD-disaggregated deployment: Flash uses a 1PID two-node form, and Pro uses a 2PID eight-node form (2 nodes per P instance, 4 per D instance). The sequence lengths are 128K input + 8K output / 8K input + 1K output. Under real data and real workloads, the throughput data at a given per-NPU batch size are shown in Table 7, and the throughput trend across input lengths is shown in Figure 7.

Prefill Latency. For Prefill, we focus on optimizing long-sequence performance. Using the low-latency deployment form (Flash model on a single A3 node, Pro model on two A3 nodes), with a

Table 7: Low-latency throughput for openPangu-2.0.

Model	ISL + OSL	TPOT (ms)	Batch Size	TPS (tokens/s per NPU)
openPangu-2.0-Flash	8K+1K	16.2	48	2963
openPangu-2.0-Flash	128K+8K	13.0	24	1846
openPangu-2.0-Pro	8K+1K	23.2	40	1727
openPangu-2.0-Pro	128K+8K	18.1	24	1326

single-batch input sequence length of 128K, under real data and real workloads, the measured Flash model TTFT is 2.6 s and the Pro model TTFT is 3.0 s; see Figure 8.

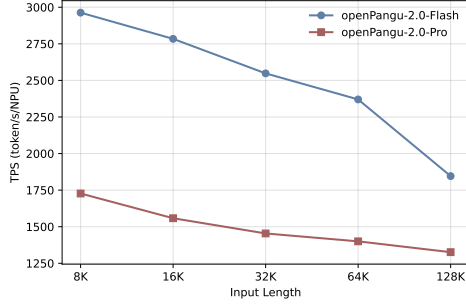


Figure 7: openPangu-2.0 Input Sequence Length vs TPS (token/s per NPU), corresponding to a latency of about 15 ms (Flash) / 20 ms (Pro).

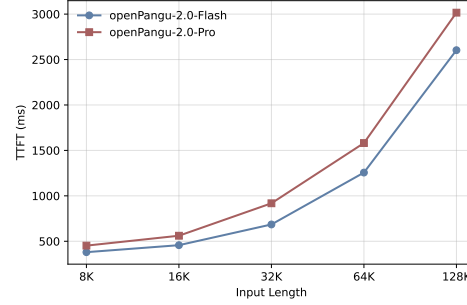


Figure 8: openPangu-2.0 TTFT vs Input Sequence Length.

7 Conclusion and Future Work

We introduced openPangu-2.0, a full-stack upgrade engineered to power agents capable of consistent, long-context reasoning. Through a rigorous hardware-software co-design methodology centered on the Ascend platform, we addressed critical bottlenecks across massive MoE training, long-horizon alignment, and inference efficiency. By synergizing architectural innovations with a dedicated inference engine that utilizes custom operator optimizations, advanced parallelism, and asynchronous scheduling, the openPangu-2.0 family achieves exceptional inference latency and throughput. Comprehensive evaluations demonstrate highly competitive performance across standard general, STEM reasoning and agentic benchmarks. Crucially, our vision extends beyond static benchmark proficiency to prioritize real-world system utility. openPangu is strategically positioned to drive the core competitiveness of agentic applications across smart devices, automotive systems, ICT infrastructure, and Huawei Cloud. Concurrently, we are actively expanding our edge-native capabilities, leveraging the Kirin silicon architecture to push the boundaries of on-device intelligence.

While foundational scaling laws continue to drive performance, the focus of scaling is shifting from static knowledge assimilation to long-trajectory, interactive data—a transition from passive models that simply *know* to agentic systems that *act*. To drive this evolution, our research roadmap remains anchored in the Ascend hardware ecosystem, where we will expand our sandbox-based synthesis pipelines to generate massive, high-fidelity execution datasets across complex, open-ended scenarios. In the coming months, through successive iterations of the model, our objective is to systematically enhance the core capabilities of openPangu in both code intelligence and complex agentic workflows. Ultimately, this execution-driven approach is designed to refine the model’s underlying cognitive architecture, producing autonomous agents capable of reasoning through and resolving multi-step workflows in dynamic, real-world environments.

References

- [1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=3zKtaqxLhW>.
- [2] Kilo AI. Pinchbench: The openclaw llm agent benchmarking leaderboard. <https://pinchbench.com/>, 2026. Accessed: 2026-07-23.
- [3] Long and Phan, Alice Gatti, Nathaniel Li, Adam Khoja, Ryan Kim, Richard Ren, Jason Hausenloy, Oliver Zhang, Mantas Mazeika, Dan Hendrycks, Ziwen Han, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, Michael Choi, Anish Agrawal, Arnav Chopra, Aakaash Nattanmai, Gordon McKellips, Anish Cheraku, Asim Suhail, Ethan Luo, Marvin Deng, Jason Luo, Ashley Zhang, Kavin Jindel, Jay Paek, Kasper Halevy, Allen Baranov, Michael Liu, Advait Avadhanam, David Zhang, Vincent Cheng, Brad Ma, Evan Fu, Liam Do, Joshua Lass, Hubert Yang, Surya Sunkari, Vishruth Bharath, Violet Ai, James Leung, Rishit Agrawal, Alan Zhou, Kevin Chen, Tejas Kalpathi, Ziqi Xu, Gavin Wang, Tyler Xiao, Erik Maung, Sam Lee, Ryan Yang, Roy Yue, Ben Zhao, Julia Yoon, Xianguan Sun, Aryan Singh, Clark Peng, Tyler Osbey, Taozhi Wang, Daryl Echeazu, Timothy Wu, Spandan Patel, Vidhi Kulkarni, Vijaykarti Sundarapandian, Andrew Le, Zafir Nasim, Srikar Yalam, Ritesh Kasamsetty, Soham Samal, David Sun, Nihar Shah, Abhijeet Saha, Alex Zhang, Leon Nguyen, Laasya Nagumalli, Kaixin Wang, Aidan Wu, Anwith Telluri, Summer Yue, Alexandr Wang, Dmitry Dodonov, Tung Nguyen, Jaeho Lee, Daron Anderson, Mikhail Doroshenko, Alun Cennyth Stokes, Mobeem Mahmood, Oleksandr Pokutnyi, Oleg Iskra, Jessica P. Wang, John-Clark Levin, Mstyslav Kazakov, Fiona Feng, Steven Y. Feng, Haoran Zhao, Michael Yu, Varun Gangal, Chelsea Zou, Zihan Wang, Serguei Popov, Robert Gerbicz, Geoff Galgon, Johannes Schmitt, Will Yeadon, Yongki Lee, Scott Sauers, Alvaro Sanchez, Fabian Giska, Marc Roth, Søren Riis, Saiteja Utpala, Noah Burns, Gashaw M. Goshu, Mohinder Maheshbhai Naiya, Chidozie Agu, Zachary Giboney, Antrell Cheatom, Francesco Fournier-Facio, Sarah-Jane Crowson, Lennart Finke, Zerui Cheng, Jennifer Zampese, Ryan G. Hoerr, Mark Nandor, Hyunwoo Park, Tim Gehrunger, Jiaqi Cai, Ben McCarty, Alexis C. Garretson, Edwin Taylor, Damien Sileo, Qiuyu Ren, Usman Qazi, Lianghui Li, Jungbae Nam, John B. Wydallis, Pavel Arkhipov, Jack Wei Lun Shi, Aras Bacho, Chris G. Willcocks, Hangrui Cao, Sumeet Motwani, Emily de Oliveira Santos, Johannes Veith, Edward Vendrow, Doru Cojoc, Kengo Zenitani, Joshua Robinson, Longke Tang, Yuqi Li, Joshua Vendrow, Natanael Wildner Fraga, Vladyslav Kuchkin, Andrey Pupasov Maksimov, Pierre Marion, Denis Efremov, Jayson Lynch, Kaiqu Liang, Aleksandar Mikov, Andrew Gritsevskiy, Julien Guillod, Gözdenur Demir, Dakotah Martinez, Ben Pageler, Kevin Zhou, Saeed Soori, Ori Press, Henry Tang, Paolo Rissone, Sean R. Green, Lina Brüssel, Moon Twayana, Aymeric Dieuleveut, Joseph Marvin Imperial, Ameya Prabhu, Jinzhou Yang, Nick Crispino, Arun Rao, Dimitri Zvonkine, Gabriel Loiseau, Mikhail Kalinin, Marco Lukas, Ciprian Manolescu, Nate Stambaugh, Subrata Mishra, Tad Hogg, Carlo Bosio, Brian P. Coppola, Julian Salazar, Jaehyeok Jin, Rafael Sayous, Stefan Ivanov, Philippe Schwaller, Shaipranesh Senthilkumar, Andres M. Bran, Andres Algaba, Kelsey Van den Houte, Lynn Van Der Sypt, Brecht Verbeken, David Noever, Alexei Kopylov, Benjamin Myklebust, Bikun Li, Lisa Schut, Evgenii Zheltonozhskii, Qiaochu Yuan, Derek Lim, Richard Stanley, Tong Yang, John Maar, Julian Wykowski, Mart Oller, Anmol Sahu, Cesare Giulio Ardito, Yuzheng Hu, Ariel Ghislain Kemogne Kamdoun, Alvin Jin, Tobias Garcia Vilchis, Yuexuan Zu, Martin Lackner, James Koppel, Gongbo Sun, Daniil S. Antonenko, Steffi Chern, Bingchen Zhao, Pierrot Arsene, Joseph M. Cavanagh, Daofeng Li, Jiawei Shen, Donato Crisostomi, Wenjin Zhang, Ali Dehghan, Sergey Ivanov, David Perrella, Nurdin Kaparov, Allen Zang, Ilia Sucholutsky, Arina Kharlamova, Daniil Orel, Vladislav Poritski, Shalev Ben-David, Zachary Berger, Parker Whitfill, Michael Foster, Daniel Munro, Linh Ho, Shankar Sivarajan, Dan Bar Hava, Aleksey Kuchkin, David Holmes, Alexandra Rodriguez-Romero, Frank Sommerhage, Anji Zhang, Richard Moat, Keith Schneider, Zakayo Kazibwe, Don Clarke, Dae Hyun Kim, Felipe Meneguitti Dias, Sara Fish, Veit Elser, Tobias Kreiman, Victor Efren Guadarrama Vilchis, Immo Klose, Ujjwala Anantheswaran, Adam Zweiger, Kaivalya Rawal, Jeffery Li, Jeremy Nguyen, Nicolas Daans, Haline Heidinger, Maksim Radionov, Václav Rozho, Vincent Ginis, Christian Stump, Niv Cohen, Rafa Powiata, Josef Tkadlec, Alan Goldfarb, Chenguang Wang, Piotr Padlewski, Stanis-

law Barzowski, Kyle Montgomery, Ryan Stendall, Jamie Tucker-Foltz, Jack Stade, T. Ryan Rogers, Tom Goertzen, Declan Grabb, Abhishek Shukla, Alan Givré, John Arnold Ambay, Archan Sen, Muhammad Fayez Aziz, Mark H. Inlow, Hao He, Ling Zhang, Younesse Kaddar, Ivar Ångquist, Yanxu Chen, Harrison K. Wang, Kalyan Ramakrishnan, Elliott Thornley, Antonio Terpin, Hailey Schoelkopf, Eric Zheng, Avishy Carmi, Ethan D. L. Brown, Kelvin Zhu, Max Bartolo, Richard Wheeler, Martin Stehberger, Peter Bradshaw, JP Heimonen, Kaustubh Sridhar, Ido Akov, Jennifer Sandlin, Yury Makarychev, Joanna Tam, Hieu Hoang, David M. Cunningham, Vladimir Goryachev, Demosthenes Patramanis, Michael Krause, Andrew Redenti, David Aldous, Jesyin Lai, Shannon Coleman, Jiangnan Xu, Sangwon Lee, Ilias Magoulas, Sandy Zhao, Ning Tang, Michael K. Cohen, Orr Paradise, Jan Hendrik Kirchner, Maksym Ovchynnikov, Jason O. Matos, Adithya Shenoy, Michael Wang, Yuzhou Nie, Anna Sztyber-Betley, Paolo Faraboschi, Robin Riblet, Jonathan Crozier, Shiv Halasyamani, Shreyas Verma, Prashant Joshi, Eli Meril, Ziqiao Ma, Jérémy Andréoletti, Raghav Singhal, Jacob Platinick, Volodymyr Nevirkovets, Luke Basler, Alexander Ivanov, Seri Khoury, Nils Gustafsson, Marco Piccardo, Hamid Mostaghimi, Qijia Chen, Virendra Singh, Tran Quoc Khanh, Paul Rosu, Hannah Szlyk, Zachary Brown, Himanshu Narayan, Aline Menezes, Jonathan Roberts, William Alley, Kunyang Sun, Arkil Patel, Max Lamparth, Anka Reuel, Linwei Xin, Hanmeng Xu, Jacob Loader, Freddie Martin, Zixuan Wang, Andrea Achilleos, Thomas Preu, Tomek Korbak, Ida Bosio, Fereshteh Kazemi, Ziyi Chen, Biró Bálint, Eve J. Y. Lo, Jiaqi Wang, Maria Inês S. Nunes, Jeremiah Milbauer, M. Saiful Bari, Zihao Wang, Behzad Ansarinejad, Yewen Sun, Stephane Durand, Hossam Elgnaiy, Guillaume Douville, Daniel Tordera, George Balabanian, Hew Wolff, Lynna Kvistad, Hsiaoyn Milliron, Ahmad Sakor, Murat Eron, Andrew Favre, Shailesh Shah, Xiaoxiang Zhou, Firuz Kamalov, Sherwin Abdoli, Tim Santens, Shaul Barkan, Allison Tee, Robin Zhang, Alessandro Tomasiello, G. Bruno De Luca, Shizhuo Looi, Vinh-Kha Le, Noam Kolt, Jiayi Pan, Emma Rodman, Jacob Drori, Carl J. Fossum, Niklas Muennighoff, Milind Jagota, Ronak Pradeep, Honglu Fan, Jonathan Eicher, Michael Chen, Kushal Thaman, William Merrill, Moritz Firsching, Carter Harris, Stefan Ciobâc, Jason Gross, Rohan Pandey, Ilya Gusev, Adam Jones, Shashank Agnihotri, Pavel Zhelnov, Mohammadreza Mofayezi, Alexander Piperski, David K. Zhang, Kostiantyn Dobarskyi, Roman Leventov, Ignat Soroko, Joshua Duersch, Vage Taamazyan, Andrew Ho, Wenjie Ma, William Held, Ruicheng Xian, Arnel Randy Zebaze, Mohanad Mohamed, Julian Noah Leser, Michelle X. Yuan, Laila Yacar, Johannes Lengler, Katarzyna Olszewska, Claudio Di Fratta, Edson Oliveira, Joseph W. Jackson, Andy Zou, Muthu Chidambaram, Timothy Manik, Hector Haffenden, Dashiell Stander, Ali Dasouqi, Alexander Shen, Bitu Golshani, David Stap, Egor Kretov, Mikalai Uzhou, Alina Borisovna Zhidkovskaya, Nick Winter, Miguel Orbegoza Rodriguez, Robert Lauff, Dustin Wehr, Colin Tang, Zaki Hossain, Shaun Phillips, Fortuna Samuele, Fredrik Ekström, Angela Hammon, Oam Patel, Faraz Farhidi, George Medley, Forough Mohammadzadeh, Madellene Peñaflor, Haile Kassahun, Alena Friedrich, Rayner Hernandez Perez, Daniel Pyda, Taom Sakal, Omkar Dhamane, Ali Khajegili Mirabadi, Eric Hallman, Kenchi Okutsu, Mike Battaglia, Mohammad Maghsoudimehrabani, Alon Amit, Dave Hulbert, Roberto Pereira, Simon Weber, Handoko, Anton Peristy, Stephen Malina, Mustafa Mehkary, Rami Aly, Frank Reidegeld, Anna-Katharina Dick, Cary Friday, Mukhwinder Singh, Hassan Shapourian, Wanyoung Kim, Mariana Costa, Hubeyb Gurdogan, Harsh Kumar, Chiara Cecconello, Chao Zhuang, Haon Park, Micah Carroll, Andrew R. Tawfeek, Stefan Steinerberger, Daattavya Aggarwal, Michael Kirchhof, Linjie Dai, Evan Kim, Johan Ferret, Jainam Shah, Yuzhou Wang, Minghao Yan, Krzysztof Burdzy, Lixin Zhang, Antonio Franca, Diana T. Pham, Kang Yong Loh, Joshua Robinson, Abram Jackson, Paolo Giordano, Philipp Petersen, Adrian Cosma, Jesus Colino, Colin White, Jacob Votava, Vladimir Vinnikov, Ethan Delaney, Petr Spelda, Vit Stritecky, Syed M. Shahid, Jean-Christophe Mourrat, Lavr Vetoshkin, Koen Sponselee, Renas Bacho, Zheng-Xin Yong, Florencia de la Rosa, Nathan Cho, Xiuyu Li, Guillaume Malod, Orion Weller, Guglielmo Albani, Leon Lang, Julien Laurendeau, Dmitry Kazakov, Fatimah Adesanya, Julien Portier, Lawrence Hollom, Victor Souza, Yuchen Anna Zhou, Julien Degorre, Yiit Yaln, Gbenga Daniel Obikoya, Rai Michael Pokorny, Filippo Bigi, M. C. Boscá, Oleg Shumar, Kaniuar Bacho, Gabriel Recchia, Mara Popescu, Nikita Shulga, Ngefor Mildred Tanwie, Thomas C. H. Lux, Ben Rank, Colin Ni, Matthew Brooks, Alesia Yakimchyk, Huanxu Quinn Liu, Stefano Cavalleri, Olle Häggström, Emil Verkama, Joshua Newbould, Hans Gundlach, Leonor Brito-Santana, Brian Amaro, Vivek Vajipey, Rynaa Grover, Ting Wang, Yosi Kratish, Wen-Ding Li, Sivakanth Gopi, Andrea Caciolai, Christian Schroeder de Witt, Pablo Hernández-Cámara, Emanuele Rodolà, Jules Robins, Dominic Williamson, Brad Raynor, Hao

Qi, Ben Segev, Jingxuan Fan, Sarah Martinson, Erik Y. Wang, Kaylie Hausknecht, Michael P. Brenner, Mao Mao, Christoph Demian, Peyman Kassani, Xinyu Zhang, David Avagian, Es-hawn Jessica Scipio, Alon Ragoler, Justin Tan, Blake Sims, Rebeka Plecnik, Aaron Kirtland, Omer Faruk Bodur, D. P. Shinde, Yan Carlos Leyva Labrador, Zahra Adoul, Mohamed Zekry, Ali Karakoc, Tania C. B. Santos, Samir Shamseldeen, Loukmane Karim, Anna Liakhovitskaia, Nate Resman, Nicholas Farina, Juan Carlos Gonzalez, Gabe Maayan, Earth Anderson, Rodrigo De Oliveira Pena, Elizabeth Kelley, Hodjat Mariji, Rasoul Pouriamanesh, Wentao Wu, Ross Finocchio, Ismail Alarab, Joshua Cole, Danyelle Ferreira, Bryan Johnson, Mohammad Safdari, Liangti Dai, Siriphan Arthornthurasuk, Isaac C. McAlister, Alejandro José Moyano, Alexey Pronin, Jing Fan, Angel Ramirez-Trinidad, Yana Malysheva, Daphiny Pottmaier, Omid Taheri, Stanley Stepanic, Samuel Perry, Luke Askew, Raúl Adrián Huerta Rodriguez, Ali M. R. Minissi, Ricardo Lorena, Krishnamurthy Iyer, Arshad Anil Fasiludeen, Ronald Clark, Josh Ducey, Matheus Piza, Maja Somrak, Eric Vergo, Juehang Qin, Benjámín Borbás, Eric Chu, Jack Lindsey, Antoine Jallon, I. M. J. McInnis, Evan Chen, Avi Semler, Luk Gloor, Tej Shah, Marc Carauleanu, Pascal Lauer, Tran Duc Huy, Hossein Shahrtash, Emilien Duc, Lukas Lewark, Assaf Brown, Samuel Albanie, Brian Weber, Warren S. Vaz, Pierre Clavier, Yiyang Fan, Gabriel Poesia Reis e Silva, Long Tony Lian, Marcus Abramovitch, Xi Jiang, Sandra Mendoza, Murat Islam, Juan Gonzalez, Vasilios Mavroudis, Justin Xu, Pawan Kumar, Laxman Prasad Goswami, Daniel Bugas, Nasser Heydari, Ferenc Jeanplong, Thorben Jansen, Antonella Pinto, Archimedes Apronti, Abdallah Galal, Ng Ze-An, Ankit Singh, Tong Jiang, Joan of Arc Xavier, Kanu Priya Agarwal, Mohammed Berkani, Gang Zhang, Zhehang Du, Benedito Alves de Oliveira Junior, Dmitry Malishev, Nicolas Remy, Taylor D. Hartman, Tim Tarver, Stephen Mensah, Gautier Abou Loume, Wiktor Morak, Farzad Habibi, Sarah Hoback, Will Cai, Javier Gimenez, Roselynn Grace Montecillo, Jakub ucki, Russell Campbell, Asankhaya Sharma, Khalida Meer, Shreen Gul, Daniel Espinosa Gonzalez, Xavier Alapont, Alex Hoover, Gunjan Chhablani, Freddie Vargus, Arunim Agarwal, Yibo Jiang, Deepakkumar Patil, David Outevsky, Kevin Joseph Scaria, Rajat Maheshwari, Abdelkader Dendane, Priti Shukla, Ashley Cartwright, Sergei Bogdanov, Niels Mündler, Sören Möller, Luca Arnaboldi, Kunvar Thaman, Muhammad Rehan Siddiqi, Prajvi Saxena, Himanshu Gupta, Tony Fruhauff, Glen Sherman, Mátyás Vincze, Siranut Usawasutsakorn, Dylan Ler, Anil Radhakrishnan, Innocent Enyekwe, Sk Md Salauddin, Jiang Muzhen, Aleksandr Maksapetyan, Vivien Rossbach, Chris Harjadi, Mohsen Bahaloohoreh, Claire Sparrow, Jasdeep Sidhu, Sam Ali, Song Bian, John Lai, Eric Singer, Justine Leon Uro, Greg Bateman, Mohamed Sayed, Ahmed Menshawy, Darling Duclosel, Dario Bezzi, Yashaswini Jain, Ashley Aaron, Murat Tiryakioglu, Sheeshram Siddh, Keith Krenek, Imad Ali Shah, Jun Jin, Scott Creighton, Denis Peskoff, Zienab EL-Wasif, Ravagendrann P, Michael Richmond, Joseph McGowan, Tejal Patwardhan, Hao-Yu Sun, Ting Sun, Nikola Zubi, Samuele Sala, Stephen Ebert, Jean Kaddour, Manuel Schottdorf, Dianzhuo Wang, Gerol Petruzella, Alex Meiburg, Tilen Medved, Ali ElSheikh, S. Ashwin Hebbar, Lorenzo Vaquero, Xianjun Yang, Jason Poulos, Vilém Zouhar, Sergey Bogdanik, Mingfang Zhang, Jorge Sanz-Ros, David Anugraha, Yinwei Dai, Anh N. Nhu, Xue Wang, Ali Anil Demircali, Zhibai Jia, Yuyin Zhou, Juncheng Wu, Mike He, Nitin Chandok, Aarush Sinha, Gaoxiang Luo, Long Le, Mickaël Noyé, Micha Perekiewicz, Ioannis Pantidis, Tianbo Qi, Soham Sachin Purohit, Letitia Parcalabescu, Thai-Hoa Nguyen, Genta Indra Winata, Edoardo M. Ponti, Hanchen Li, Kaustubh Dhole, Jongee Park, Dario Abbondanza, Yuanli Wang, Anupam Nayak, Diogo M. Caetano, Antonio A. W. L. Wong, Maria del Rio-Chanona, Dániel Kondor, Pieter Francois, Ed Chilstrey, Jakob Zsambok, Dan Hoyer, Jenny Reddish, Jakob Hauser, Francisco-Javier Rodrigo-Ginés, Suchandra Datta, Maxwell Shepherd, Thom Kamphuis, Qizheng Zhang, Hyunjun Kim, Ruiji Sun, Jianzhu Yao, Franck Deroncourt, Satyapriya Krishna, Sina Rismanchian, Bonan Pu, Francesco Pinto, Yingheng Wang, Kumar Shridhar, Kalon J. Overholt, Glib Briia, Hieu Nguyen, David Quod Soler Bartomeu, Tony CY Pang, Adam Wecker, Yifan Xiong, Fanfei Li, Lukas S. Huber, Joshua Jaeger, Romano De Maddalena, Xing Han Lù, Yuhui Zhang, Claas Beger, Patrick Tser Jern Kon, Sean Li, Vivek Sanker, Ming Yin, Yihao Liang, Xinlu Zhang, Ankit Agrawal, Li S. Yifei, Zechen Zhang, Mu Cai, Yasin Sonmez, Costin Cozianu, Changhao Li, Alex Slen, Shoubin Yu, Hyun Kyu Park, Gabriele Sarti, Marcin Briaski, Alessandro Stolfo, Truong An Nguyen, Mike Zhang, Yotam Perlitz, Jose Hernandez-Orallo, Runjia Li, Amin Shabani, Felix Juefei-Xu, Shikhar Dhingra, Orr Zohar, My Chiffon Nguyen, Alexander Pondaven, Abdurrahim Yilmaz, Xuandong Zhao, Chuanyang Jin, Muyan Jiang, Stefan Todoran, Xinyao Han, Jules Kreuer, Brian Rabern, Anna Plassart, Martino Maggetti, Luther Yap, Robert Geirhos, Jonathon Kean, Dingsu Wang, Sina Mollaei, Chenkai Sun, Yifan Yin,

- Shiqi Wang, Rui Li, Yaowen Chang, Anjiang Wei, Alice Bizeul, Xiaohan Wang, Alexandre Oliveira Arrais, Kushin Mukherjee, Jorge Chamorro-Padial, Jiachen Liu, Xingyu Qu, Junyi Guan, Adam Bouyamourn, Shuyu Wu, Martyna Plomecka, Junda Chen, Mengze Tang, Jiaqi Deng, Shreyas Subramanian, Haocheng Xi, Haoxuan Chen, Weizhi Zhang, Yinuo Ren, Haoqin Tu, Sejong Kim, Yushun Chen, Sara Vera Marjanovi, Junwoo Ha, Grzegorz Luczyna, Jeff J. Ma, Zewen Shen, Dawn Song, Cedegao E. Zhang, Zhun Wang, Gaël Gendron, Yunze Xiao, Leo Smucker, Erica Weng, Kwok Hao Lee, Zhe Ye, Stefano Ermon, Ignacio D. Lopez-Miguel, Theo Knights, Anthony Gitter, Namkyu Park, Boyi Wei, Hongzheng Chen, Kunal Pai, Ahmed Elkhany, Han Lin, Philipp D. Siedler, Jichao Fang, Ritwik Mishra, Károly Zsolnai-Fehér, Xilin Jiang, Shadab Khan, Jun Yuan, Rishab Kumar Jain, Xi Lin, Mike Peterson, Zhe Wang, Aditya Malusare, Maosen Tang, Isha Gupta, Ivan Fosin, Timothy Kang, Barbara Dworakowska, Kazuki Matsumoto, Guangyao Zheng, Gerben Sewuster, Jorge Pretel Villanueva, Ivan Rannev, Igor Chernyavsky, Jiale Chen, Deepayan Banik, Ben Racz, Wen-chao Dong, Jianxin Wang, Laila Bashmal, Duarte V. Gonçalves, Wei Hu, Kaushik Bar, Ondrej Bohdal, Atharv Singh Patlan, Shehzaad Dhuliawala, Caroline Geirhos, Julien Wist, Yuval Kansal, Bingsen Chen, Kutay Tire, Atak Talay Yücel, Brandon Christof, Veerupaksh Singla, Zijian Song, Sanxing Chen, Jiaxin Ge, Kaustubh Ponkshe, Isaac Park, Tianneng Shi, Martin Q. Ma, Joshua Mak, Sherwin Lai, Antoine Moulin, Zhuo Cheng, Zhanda Zhu, Ziyi Zhang, Vaidehi Patil, Ketan Jha, Qitong Men, Jiaxuan Wu, Tianchi Zhang, Bruno Hebling Vieira, Alham Fikri Aji, Jae-Won Chung, Mohammed Mahfoud, Ha Thi Hoang, Marc Sperzel, Wei Hao, Kristof Meding, Sihan Xu, Vassilis Kostakos, Davide Manini, Yueying Liu, Christopher Toukmaji, Eunmi Yu, Arif Engin Demircali, Zhiyi Sun, Ivan Dewerpe, Hongsen Qin, Roman Pflugfelder, James Bailey, Johnathan Morris, Ville Heilala, Sybille Rosset, Zishun Yu, Peter E. Chen, Woongyeong Yeo, Eeshaan Jain, Sreekar Chigurupati, Julia Chernyavsky, Sai Prajwal Reddy, Subhashini Venugopalan, Hunar Batra, Core Francisco Park, Hieu Tran, Guilherme Maximiano, Genghan Zhang, Yizhuo Liang, Hu Shiyu, Rongwu Xu, Rui Pan, Siddharth Suresh, Ziqi Liu, Samaksh Gulati, Songyang Zhang, Peter Turchin, Christopher W. Bartlett, Christopher R. Scotese, Phuong M. Cao, Ben Wu, Jacek Karwowski, and Davide Scaramuzza. A benchmark of expert-level academic questions to assess ai capabilities. *Nature*, 649(8099): 1139–1146, January 2026. ISSN 1476-4687. doi: 10.1038/s41586-025-09962-4. URL <http://dx.doi.org/10.1038/s41586-025-09962-4>.
- [4] Mislav Balunovi, Jasper Dekoninck, Ivo Petrov, Nikola Jovanovi, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions, 2026. URL <https://arxiv.org/abs/2505.23281>.
 - [5] Chaithanya Bandi, Razvan-Gabriel Dumitru, Ben Hertzberg, Divyansh Agarwal, Geobio Boo, Tejas Polakam, Sami Hassaan, Jeff Da, HiJae Kim, Vipul Gupta, Manasi Sharma, Andrew Park, Martin Dimakis, Ernesto Gabriel Hernandez Montoya, Dan Rambado, Ivan Salazar, Rafael Cruz, MohammadHossein Rezaei, Chetan Rane, Ben Levin, Daniel Yue Zhang, Brad Kenstler, and Bing Liu. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers, 2026. URL <https://arxiv.org/abs/2602.00933>.
 - [6] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment, 2025. URL <https://arxiv.org/abs/2506.07982>.
 - [7] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL <https://arxiv.org/abs/2004.05150>.
 - [8] Haorui Chen, Chengze Li, and Jia Li. Featbench: Towards more realistic evaluation of feature-level code generation, 2026. URL <https://arxiv.org/abs/2509.22237>.
 - [9] Alireza Daghighfarsoodeh, Chung-Yu Wang, Hamed Taherkhani, Melika Sepidband, Mohammad Abdollahi, Hadi Hemmati, and Hung Viet Pham. Deep-bench: Deep learning benchmark dataset for code generation, 2025. URL <https://arxiv.org/abs/2502.18726>.
 - [10] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

- [11] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [12] DeepSeek-AI. Deepseek-v3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- [13] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024. URL <https://arxiv.org/abs/2405.04434>.
- [14] DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL <https://arxiv.org/abs/2512.02556>.
- [15] DeepSeek-AI, Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chengyu Hou, Chenhao Xu, Chenze Shao, Chong Ruan, Conner Sun, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Donghao Li, Dongjie Ji, Erhang Li, Fang Wei, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei, Haotian Yuan, Haowei Zhang, Haowen Luo, Haoyu Chen, Haozhe Ji, Hengqing Zhang, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J Yang, JQ Zhu, Jia Luo, Jia Song, Jia Yu, Jialiang Huang, Jialu Cai, Jian Liang, Jiangting Zhou, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jieyu Yang, Jin Chen, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jingzi Zhou, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junmin Zheng, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Leyi Xia, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Linyan Zhu, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, MS Di, MY Xu, Max Mei, Miaoqun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Mingxu Zhou, Minmin Han, Ning Wang, Panpan Huang, Panpan Wang, Peixin Cong, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, Ruyi Chen, SH Liu, Shanghao Lu, Shangmian Sun, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoqing Wu, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W Zhang, WL Xiao, Wangding Zeng, Wei An, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlve Huang, Wenqing Hou, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyu Zhang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xueyin Chen, Xuheng Lin, Xuwei Fu, YC Yan, YQ Wang, YW Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Xu, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Shao, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Yiling Ma, Ying He, Ying Tang, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, YuKun Li, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yudian Wang, Yuehan Yang, Yuer Xu, Yuhao Wu, Yuhao Meng, Yuheng Zou, Yukun Zha, Yunfan Xiong, Yupeng Chen, Yuping Lin, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yuting Yan, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, ZF Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zekai Zhang, Zhangli Sha, Zhe Fu, Zhe Ju, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zheren Gao, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhongyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihua Qu, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Ziyi Wan, Zizheng Pan, and Zongqing Yao. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026. URL <https://arxiv.org/abs/2606.19348>.

- [16] Jasper Dekoninck, Nikola Jovanovi, Tim Gehrunger, Kári Rognvaldsson, Ivo Petrov, Chenhao Sun, and Martin Vechev. Beyond benchmarks: Matharena as an evaluation platform for mathematics with llms. 2026. URL <https://arxiv.org/abs/2605.00674>.
- [17] Kaustubh Deshpande, Ved Sirdeshmukh, Johannes Baptist Mols, Lifeng Jin, Ed-Yeremai Hernandez-Cardona, Dean Lee, Jeremy Kritz, Willow E Primack, Summer Yue, and Chen Xing. Multichallenge: A realistic multi-turn conversation evaluation benchmark challenging to frontier llms. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 18632–18702, 2025.
- [18] Ming Ding, Zhuoyi Yang, Wenyi Hong, Wendi Zheng, Chang Pei, Bo Peng, Yinchu Ding, Jie Tang, and Yuxiao Dong. Cogview: Mastering text-to-image generation via transformers. In *Advances in Neural Information Processing Systems*, volume 34, pages 19822–19835, 2021.
- [19] Shuangrui Ding, Xuanlang Dai, Long Xing, Shengyuan Ding, Ziyu Liu, Yang JingYi, Penghui Yang, Zhixiong Zhang, Xilin Wei, Xinyu Fang, et al. Wildclawbench: A benchmark for real-world, long-horizon agent evaluation. *arXiv preprint arXiv:2605.10912*, 2026.
- [20] Shihan Dou, Ming Zhang, Zhangyue Yin, Chenhao Huang, Yujiong Shen, Junzhe Wang, Jiayi Chen, Yuchen Ni, Junjie Ye, Cheng Zhang, et al. Cl-bench: A benchmark for context learning. *arXiv preprint arXiv:2602.03587*, 2026.
- [21] GLM-5 Team et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.
- [22] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction, 2024. URL <https://arxiv.org/abs/2404.19737>.
- [23] Xiangming Gu, Tianyu Pang, Chao Du, Qian Liu, Fengzhuo Zhang, Cunxiao Du, Ye Wang, and Min Lin. When attention sink emerges in language models: An empirical view, 2025. URL <https://arxiv.org/abs/2410.10781>.
- [24] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=5h0qf71BZZ>.
- [25] X Guo, Zhenyu Huang, Zujie Wang, Wen, and et al. Zhang. Kpop: Taming training–inference mismatch in reinforcement learning with adaptive masking regions. 2026. URL <https://ringtech.notion.site/Taming-Training-Inference-Mismatch-in-Reinforcement-Learning-with-Adaptive-Masking-Regions-364c8705a032801b84dcc4c5f7682549>.
- [26] Lukas Haas, Gal Yona, Giovanni D’Antonio, Sasha Goldshtein, and Dipanjan Das. Simpleqa verified: A reliable factuality benchmark to measure parametric knowledge. *arXiv preprint arXiv:2509.07968*, 2025.
- [27] Yancheng He, Shilong Li, Jiaheng Liu, Yingshui Tan, Weixun Wang, Hui Huang, Xingyuan Bu, Hangyu Guo, Chengwei Hu, Boren Zheng, Zhuoran Lin, Xuepeng Liu, Dekai Sun, Shirong Lin, Zhicheng Zheng, Xiaoyong Zhu, Wenbo Su, and Bo Zheng. Chinese simpleqa: A chinese factuality evaluation for large language models, 2024. URL <https://arxiv.org/abs/2411.07140>.
- [28] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models, 2023.
- [29] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.

- [30] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- [31] Mehran Kazemi, Bahare Fatemi, Hritik Bansal, John Palowitch, Chrysovalantis Anastasiou, Sanket Vaibhav Mehta, Lalit K. Jain, Virginia Aglietti, Disha Jindal, Peter Chen, Nishanth Dikkala, Gladys Tyen, Xin Liu, Uri Shalit, Silvia Chiappa, Kate Olszewska, Yi Tay, Vinh Q. Tran, Quoc V. Le, and Orhan Firat. Big-bench extra hard, 2025. URL <https://arxiv.org/abs/2502.19187>.
- [32] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [33] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, and Han chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026. URL <https://arxiv.org/abs/2602.12670>.
- [34] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for llm training, 2025. URL <https://arxiv.org/abs/2502.16982>.
- [35] Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. Towards robust mathematical reasoning, 2025. URL <https://arxiv.org/abs/2511.01846>.
- [36] Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing moe reinforcement learning by aligning training and inference routers. *arXiv preprint arXiv:2510.11370*, 2025.
- [37] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 561–577, Carlsbad, CA, October 2018. USENIX Association. ISBN 978-1-939133-08-3. URL <https://www.usenix.org/conference/osdi18/presentation/moritz>.
- [38] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL <https://arxiv.org/abs/2508.10925>.
- [39] Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simón Posada Fishman, Marwan Aljube, Phoebe Thacker, Laurance Fauconnet, Natalie S. Kim, Patrick Chao, Samuel Miserendino, Gildas Chabot, David Li, Michael Sharman, Alexandra Barr, Amelia Glaese, and Jerry Tworek. Gdpval: Evaluating ai model performance on real-world economically valuable tasks, 2025. URL <https://arxiv.org/abs/2510.04374>.
- [40] Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hanna Hajishirzi. Generalizing verifiable instruction following. *Advances in Neural Information Processing Systems*, 38, 2026.
- [41] Yunjia Qi, Hao Peng, Xiaozhi Wang, Amy Xin, Youfeng Liu, Bin Xu, Lei Hou, and Juanzi Li. Agentif: Benchmarking instruction following of large language models in agentic scenarios. *arXiv preprint arXiv:2505.16944*, 2025.
- [42] Yanzhao Qin, Tao Zhang, Yanjun Shen, Wenjing Luo, Haoze Sun, Yan Zhang, Yujing Qiao, Weipeng Chen, Zenan Zhou, Wentao Zhang, et al. Sysbench: Can large language models follow system messages? *arXiv preprint arXiv:2408.10943*, 2024.

- [43] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020. doi: 10.1109/SC41405.2020.00024.
- [44] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- [45] Wei Shao, Zheng Lingchao, Pengyu Wang, Peizhen Zheng, Li Jun, and Yuwei Fan. Lopt: Lossless parallel tokenization acceleration for long context inference of large language model. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33107–33122, 2026.
- [46] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [47] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- [48] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [49] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2021. URL <https://arxiv.org/abs/2104.09864>.
- [50] Mingjie Sun, Xinlei Chen, J. Zico Kolter, and Zhuang Liu. Massive activations in large language models, 2024. URL <https://arxiv.org/abs/2402.17762>.
- [51] Gemma Team. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- [52] Ling Team et al. Every step evolves: Scaling reinforcement learning for trillion-scale thinking model. *arXiv preprint arXiv:2510.18855*, 2025.
- [53] Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts, 2024. URL <https://arxiv.org/abs/2408.15664>.
- [54] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents, 2025. URL <https://arxiv.org/abs/2504.12516>.
- [55] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pages 38087–38099. PMLR, 2023.
- [56] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2023. URL <https://arxiv.org/abs/2309.17453>.
- [57] Zhenda Xie, Yixuan Wei, Huanqi Cao, Chenggang Zhao, Chengqi Deng, Jiashi Li, Damai Dai, Huazuo Gao, Jiang Chang, Kuai Yu, et al. mhc: Manifold-constrained hyper-connections. *arXiv preprint arXiv:2512.24880*, 2025.
- [58] Zhenda Xie, Yixuan Wei, Huanqi Cao, Chenggang Zhao, Chengqi Deng, Jiashi Li, Damai Dai, Huazuo Gao, et al. mhc: Manifold-constrained hyper-connections, 2025. URL <https://arxiv.org/abs/2512.24880>.

- [59] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [60] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule, 2025. URL <https://arxiv.org/abs/2412.06464>.
- [61] Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, Qi Liu, Zhifang Sui, and Tong Yang. Claw-eval: Towards trustworthy evaluation of autonomous agents, 2026. URL <https://arxiv.org/abs/2604.06132>.
- [62] Yichun Yin, Wenyong Huang, Kaikai Song, Yehui Tang, Xueyu Wu, Wei Guo, Peng Guo, Yaoyuan Wang, Xiaojun Meng, Yasheng Wang, Dong Li, Can Chen, Dandan Tu, Yin Li, Fisher Yu, Ruiming Tang, Yunhe Wang, Baojun Wang, Bin Wang, Bo Wang, Boxiao Liu, Changzheng Zhang, Duyu Tang, Fei Mi, Hui Jin, Jiansheng Wei, Jiarui Qin, Jinpeng Li, Jun Zhao, Liqun Deng, Lin Li, Minghui Xu, Naifu Zhang, Nianzu Zheng, Qiang Li, Rongju Ruan, Shengjun Cheng, Tianyu Guo, Wei He, Wei Li, Weiwen Liu, Wulong Liu, Xinyi Dai, Yonghan Dong, Yu Pan, Yue Li, Yufei Wang, Yujun Li, Yunsheng Ni, Zhe Liu, Zhenhe Zhang, and Zhicheng Liu. Pangu ultra: Pushing the limits of dense large language models on ascend npus, 2025. URL <https://arxiv.org/abs/2504.07866>.
- [63] Yifan Zhang and Team Math-AI. American invitational mathematics examination (aime) 2026, 2026.
- [64] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.
- [65] Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. Hyper-connections, 2024. URL <https://arxiv.org/abs/2409.19606>.
- [66] 苏剑林. Moe 环游记: 3、换个思路来分配. 科学空间 (kexue.fm), 2025. URL <https://spaces.ac.cn/archives/10757>.

A Appendix / supplemental material

A.1 Architecture Hyperparameters

The detailed architecture hyperparameters for the openPangu-2.0-Flash and openPangu-2.0-Pro models are provided in Table 8. We also provide the hyperparameters for a MoE15B-A2B model, which we employ for preliminary ablation studies and fast proof-of-concept experiments.

Table 8: Architecture hyperparameters for the openPangu-2.0 series.

Hyperparameter	MoE15B-A2B	OpenPangu-2.0-Flash	OpenPangu-2.0-Pro
Total Parameters	15B	92B	505B
Activated Parameters	2B	6B	18B
Total Layers	32	46	50
Dense Layers	2	2	3
MoE Layers	30	44	47
Hidden Size	1536	2560	5120
Vocabulary Size	151552	151552	151552
Attention Heads	48	48	64
Query Projection Rank	384	1024	1536
Key-Value Projection Rank	512	512	512
Non-Positional QK Dimensions	128	128	128
RoPE Dimensions	64	64	64
Value Head Dimensions	128	128	128
Dense FFN Intermediate Size	7680	9216	16128
Routed Experts	128	256	384
Shared Experts	1	1	1
Activated Routed Experts	8	8	8
Expert Intermediate Size	768	1024	1792
MTP Layers	1	3	3
DSA:SWA Ratio	1:2	1:2	1:2
SWA Window Size	512	512	512

A.2 Parametric Attention Sinks vs. gpt-oss Sink

We conduct pilot experiments to evaluate PAS. All experiments use the MoE15B-A2B model baseline, trained on 250B tokens. First, PAS yields a smoother gradient norm throughout training, as shown in Figure 9: adding parametric sinks noticeably reduces gradient-norm spikes.

We further compare training loss on a hybrid SWA model with a FULL : SWA = 1:2 configuration, as shown in Figure 10. While adding parametric sinks brings no clear loss benefit for a full attention baseline, it yields a consistent training-loss gain for the hybrid SWA model, and outperforms the sink used in gpt-oss by a clear margin. We attribute this to the local nature of SWA layers: without an explicit sink, attention mass must still be distributed among real tokens inside the window even when none are worth attending to, which pushes some tokens into local-sink roles and destabilizes the attention distribution. The parametric sinks provide SWA layers with a fixed absorbing channel, offering a stable global sink for the FULL/DSA layers and a fallback attention target for the SWA layers when the local window lacks informative content.

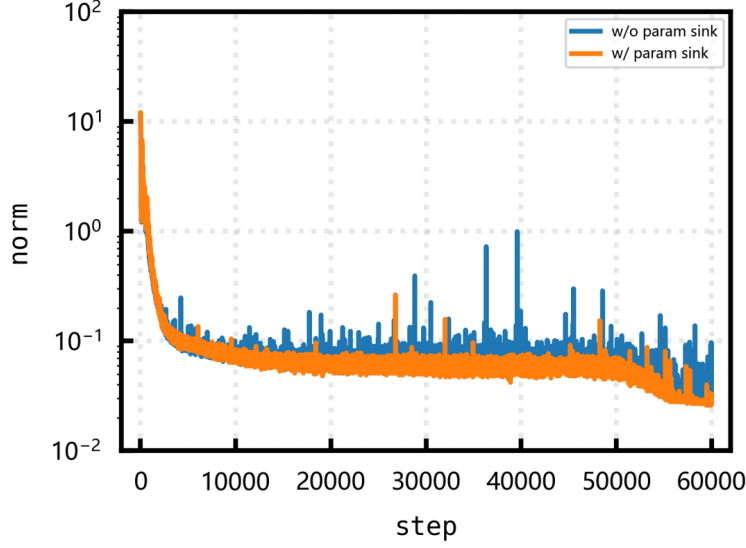


Figure 9: Gradient norm during training with and without parametric sinks (PAS). Adding parametric sinks produces a smoother gradient norm with fewer spikes.

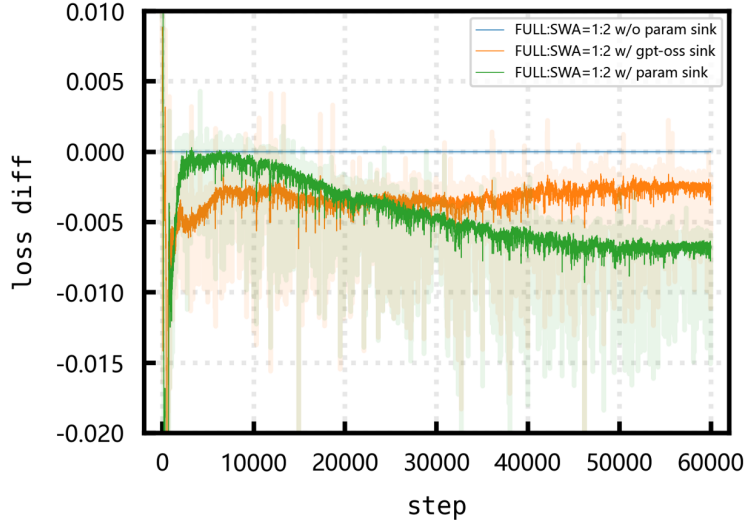


Figure 10: Training-loss difference for a FULL : SWA = 1:2 hybrid SWA model, comparing no parametric sink (conventional attention), the gpt-oss sink, and the parametric attention sinks (PAS). Curves show the loss difference relative to the no-parametric-sink baseline (lower is better).

A.3 Adaptive Auxiliary-Loss-Free Balancing

To evaluate the adaptive auxiliary-loss-free load balancing strategy discussed in Section 2.2, we compare its training dynamics against the original auxiliary-loss-free bias (which uses a sign-based update). As illustrated in Figure 11, both methods maintain the same convergence rate, yielding nearly identical LM loss under the same configuration. However, the adaptive RMS-normalized update achieves stronger load control: it consistently attains a lower batch-level load-balance violation (MaxVio), and this advantage widens as training progresses.

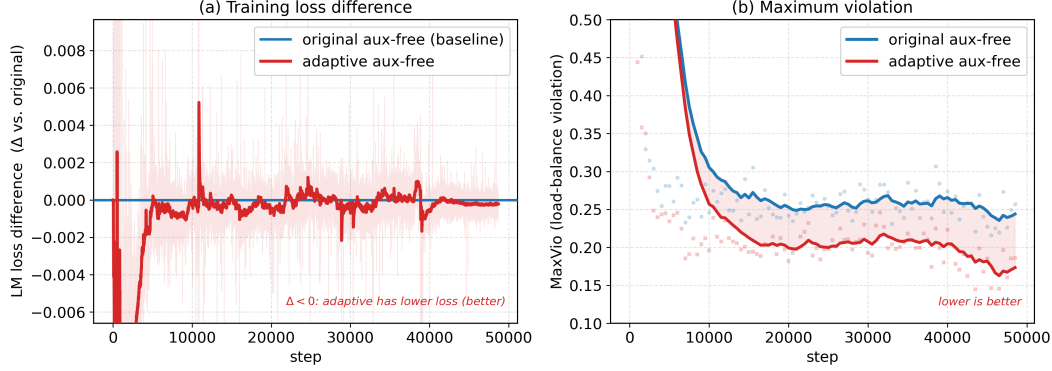


Figure 11: Training comparison between the adaptive auxiliary-loss-free bias (RMS-normalized update) and the original auxiliary-loss-free bias (sign update). (a) LM loss difference relative to the original baseline; (b) batch-level load-balance violation (MaxVio, lower is better).

A.4 Newton–Schulz Iteration Coefficients

When orthogonalizing the update matrix with the Newton–Schulz iteration in the Muon optimizer, we run 5 iterations with the per-iteration coefficients (a, b, c) listed in Table 9.

Table 9: Per-iteration Newton–Schulz coefficients used by the Muon optimizer³.

Iteration	a	b	c
1	4.0848	−6.8946	2.9270
2	3.9505	−6.3029	2.6377
3	3.7418	−5.5913	2.3037
4	2.8769	−3.1427	1.2046
5	2.8366	−3.0525	1.2012

A.5 Attention Logits Stability with Muon

Prior work [34] reports that training large models with the Muon optimizer can cause the attention logits (i.e., $QK^\top$) to explode, which in turn harms training stability, and addresses this issue with a QK-clip technique. Throughout the entire training of openPangu-2.0, however, the attention logits remain stable. As shown in Figure 12, which tracks the full training run of openPangu-2.0-Pro, the maximum attention logits stay within a bounded range (around 20) without experiencing explosive growth. We hypothesize that this stability naturally arises from architectural components such as sandwich normalization and block post-normalization, which effectively control the scale of hidden states and outputs. We leave a more systematic ablation of this phenomenon to future work.

³Coefficients are adopted from https://github.com/KellerJordan/modded-nanogpt/blob/0674386070ceb4dcd207e1aca747ffcea6c15250/train_gpt_medium.py#L45

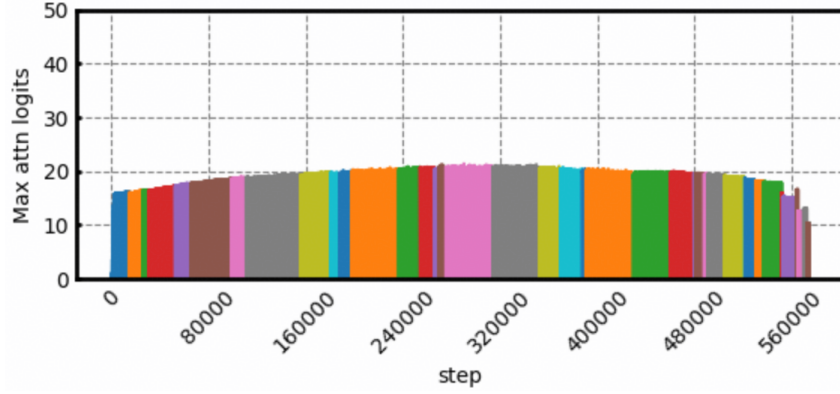


Figure 12: Maximum attention logits over the full training run of openPangu-2.0-Pro. The attention logits stay within a bounded range throughout training.

A.6 Summary of Operator-Level Mismatch

We summarize the training- and inference-side implementation differences in Table 10, which captures both the *operator-level* inconsistency (the Train-Infer Cons.) and the *input-shape* inconsistency (the B/S Cons.).

Table 10: Training-inference operator consistency across modules

Module	Training Operator	Inference Operator	B/S Cons.	Train-Infer Cons.
MatMul	torch.Linear	torch.Linear	No	Yes
	torch.matmul	torch.matmul	No	Yes
	torch.matmul	TransposeBatchMatMul	No	No
	torch_npu.npu_grouped_matmul	torch_npu.npu_grouped_matmul	Yes	Yes
RMSNorm	torch_npu.npu_rms_norm	torch_npu.npu_rms_norm	Yes	Yes
	torch_npu.npu_rms_norm	npu_ai_infra_kv_rmsnorm_rope_cache_v2	No	No
	vanilla torch ops	torch_npu.npu_add_rms_norm	Yes	No
RoPE	torch_npu.npu_rotary_mul	torch_npu.npu_rotary_mul	Yes	Yes
	torch_npu.npu_rotary_mul	npu_ai_infra_kv_rmsnorm_rope_cache_v2	Yes	Yes
	vanilla torch ops	torch_npu.npu_apply_rotary_pos_emb	Yes	Yes
	vanilla torch ops	torch_npu.npu_interleave_rope	Yes	Yes
	vanilla torch ops	torch_npu.npu_mrope	Yes	Yes
Attention	npu_sparse_flash_attention_enhance (sfa) + torch_npu.npu_fusion_attention (sink)	npu_ai_infra_sparse_flash_attention_pioneer	No	No
	torch_npu.npu_fusion_attention (mla)	npu_fused_infer_attention_sink	No	No
mHC	npu_manifold_constrained_hyper_connection_pre npu_sinkhorn	npu_manifold_constrained_hyper_connection_pre npu_sinkhorn	No	Yes
Comm	torch.distributed.all_reduce	torch.distributed.all_reduce	No	Yes
	torch.distributed.reduce_scatter	torch.distributed.reduce_scatter	No	Yes
MoE	vanilla torch ops	torch_npu.npu_moe_init_routing torch_npu.npu_moe_init_routing_v2	Yes	Yes
	vanilla torch ops	torch_npu.npu_moe_re_routing	Yes	Yes
	vanilla torch ops, experts sorted by ID, fp32	torch_npu.npu_moe_distribute_combine	Yes	No
	vanilla torch ops, experts sorted by ID, fp32	torch_npu.npu_moe_finalize_routing	Yes	No
	vanilla torch ops	torch_npu.npu_moe_gating_top_k	Yes	Yes

A.7 Methodologies for Full Alignment between Training and Inference

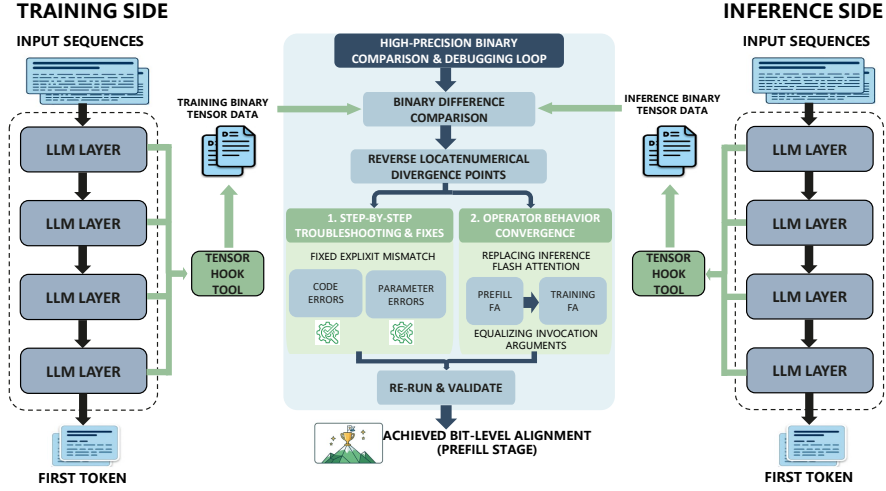


Figure 13: Strict alignment between training forward and inference prefill.

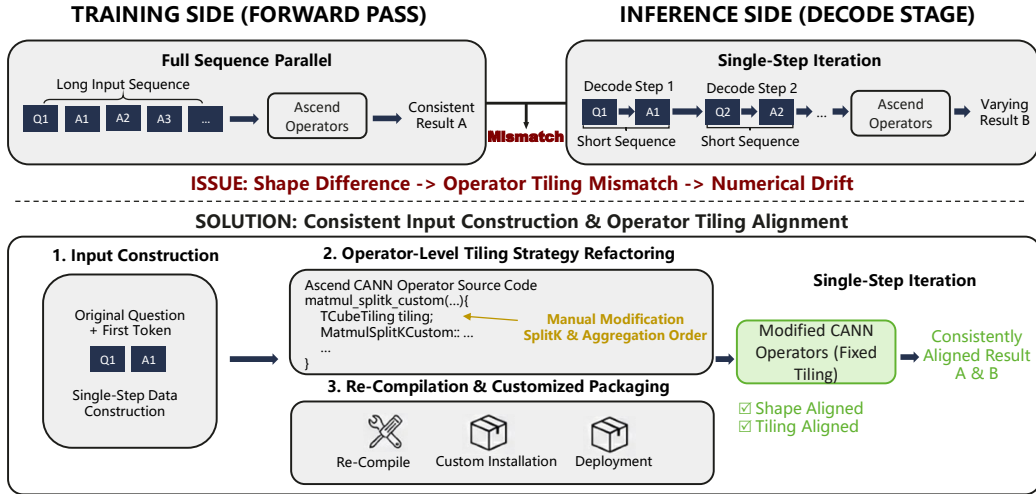


Figure 14: Operator alignment between training forward and inference decode. To eliminate the discrepancies caused by batch and sequence mismatch, we construct consistent inputs and align the operator tiling by manual modification for SplitK and aggregation order. The custom CANN operators lead to fully aligned results from training and inference.

A.8 Per-module Quantization Decision

Table 11: Per-module quantization decision for the Pro configuration (linear-layer parameter counts are per-layer values; MoE expert values are per-expert, with 384 experts in total; embedding/output heads are global weights of about 0.78 B each; MHC is a per-layer module with negligible parameter count).

Module	Operator	Params	W/A	Rationale
MLA	o_proj	41.9 M	INT8/INT8	Largest linear layer on the attention path ($\approx 52\%$)
	q_b_proj	18.9 M	INT8/INT8	Large matrix; downstream softmax tolerates quantization noise
	kv_b_proj	8.39 M	BF16/BF16	Stores W_{UK}/W_{UV} ; the same weight participates in matmuls in both W and W^T orientations, which per-token or per-channel quantization cannot satisfy simultaneously
	q_a_proj	7.86 M	BF16/BF16	Small matrix; attention entry, error propagates along the residual stream
	kv_a_proj	2.95 M	BF16/BF16	Small matrix; attention entry
LI	wq_b	6.29 M	BF16/BF16	Upstream of TopK discrete selection; weight quantization noise would flip boundary routing, so BF16 is kept
	wk	0.66 M	BF16/BF16	Tiny matrix, upstream of TopK, kept BF16
	weights_proj	0.16 M	BF16/BF16	Matrix too small; quantization gain does not offset the quantization-operator overhead
Dense MLP	gate_proj	82.6 M	INT8/INT8	First three dense MLP layers, large matrices
	up_proj	82.6 M	INT8/INT8	Same as above
	down_proj	82.6 M	INT8/INT8	Same as above
MoE	gate_proj	9.2 M/ea	INT8/INT8	Per-expert independent calibration; gate/up share the same smooth scale s ; routed experts are the main body of INT8 compression (≈ 10.6 B per layer, hence listed first in this group)
	up_proj	9.2 M/ea	INT8/INT8	Same as above
	down_proj	9.2 M/ea	INT8/INT8	The smooth scale $s_{\text{down},i}$ of down is independent per expert
	shared expert	27.5 M	BF16/BF16	Traversed by every token, error accumulates globally, and its latency is already hidden by the side stream; weight still multiplied by s for compensation
	mlp.gate	1.97 M	BF16/BF16	TopK(top-8/384) discrete selection, sensitive to perturbation—quantization noise would flip routing; only $\approx 0.02\%$ of parameters, negligible quantization gain
Other	embed_tokens	0.78 B	BF16/BF16	Embedding lookup, no gain from quantization
	lm_head / shared_head	0.78 B	BF16/BF16	Output head, directly determine logits, precision-sensitive
	MHC	—	BF16/BF16	Dynamic range becomes extreme after Sinkhorn normalization

A.9 LoPT: Lossless Parallel Tokenization

As context length grows (up to 1M tokens), tokenization becomes a non-negligible bottleneck. Mainstream algorithms (BPE, WordPiece) are inherently sequential—a token’s splitting depends on its surrounding context. A natural acceleration idea is to split the text into chunks, tokenize them in parallel, and concatenate. But this faces the Boundary Artifact: the same substring may tokenize differently at different positions, so results from separately tokenizing adjacent chunks cannot align with the whole. Existing solutions—delimiter-based splitting (low accuracy) and overlap-based splitting with Token ID matching (identical IDs do not guarantee the same character position)—cannot reach 100% accuracy.

In our system, this inconsistency hurts not only model quality but also inference speed, because we deeply integrate Token-level APC (Automatic Prefix Caching), which reuses KV Cache by precisely

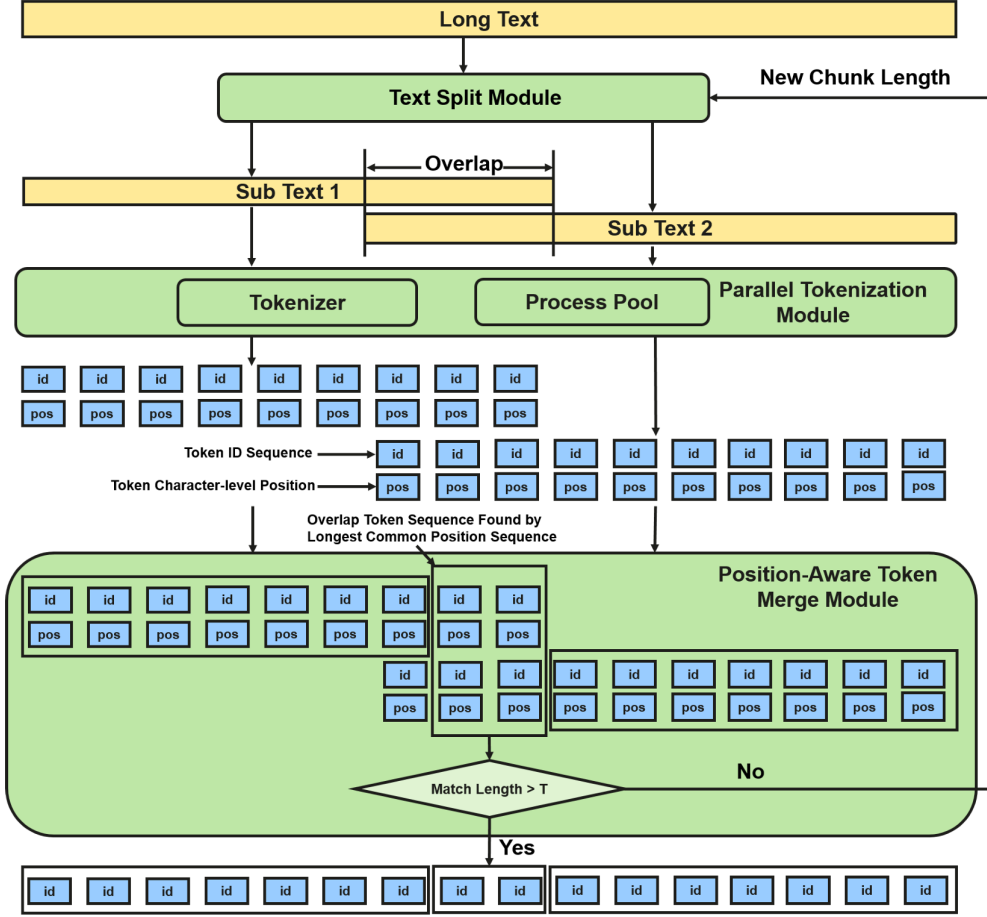


Figure 15: LoPT architecture overview. Three core modules—Text Split Module (with overlapping splits and dynamic chunk adjustment), Parallel Tokenization Module (multi-process pool for parallel tokenization, outputting token ID + character position), and Position-Aware Token Merge Module (matching and concatenation by character position, with chunk-doubling retry on match failure).

matching input Token ID sequences. If parallel tokenization introduces boundary artifacts, even a single-token misalignment causes APC hash matching to fail completely, forcing KV recomputation that can offset all tokenization gains and even degrade end-to-end performance.

To achieve lossless parallel tokenization, we propose LoPT [45]. Its core innovation: while retaining overlapping splits, it replaces “Token ID matching” with “character-position matching”. LoPT records each token’s character start/end positions in the original text and matches by position rather than ID—matching “identity” (position) rather than “category” (ID), fundamentally resolving context-induced ambiguity. With a dynamic chunk-length adjustment mechanism, LoPT produces output strictly identical to serial tokenization while achieving near-parallel-strategy speedup.

Figure 15 shows the three modules: text splitting (chunks with overlap), parallel tokenization (multi-process), and position-aware token merging (concatenation by character position). We verified: (1) Losslessness—output strictly identical to serial tokenization, 100% accuracy across code, natural language, and multilingual texts; (2) Speedup—3–5 $\times$ over serial on a 16-core CPU; (3) Scalability—stable speedup as text grows, overlap overhead below 5%. LoPT is integrated into our inference framework, improving long-context tokenization without affecting model accuracy or APC hit rate.